
tracketch Documentation

Release 1.0.0

**Jeppe Brage Christensen
Paul Scherrer Institute (PSI)**

May 04, 2026

USER GUIDE

1	Getting Started	3
1.1	Installation	3
1.2	Quick example	3
1.3	Etch-rate model selection	4
1.4	Particles and stopping-power sources	4
1.5	Target material	5
1.6	Simulation grid	5
1.7	Building the documentation	6
2	Examples	7
2.1	Simulating track shapes	7
2.2	Hydrogen isotopes – proton, deuteron, triton	8
2.3	Radial dose distributions	10
2.4	LET and CSDA range	11
2.5	Customising the simulation grid	12
2.6	Comparing arrival-time solvers	13
3	Algorithms	15
3.1	Simulation pipeline	15
3.2	Arrival-time solvers	15
3.3	Etch-rate model	16
4	API Reference	17
4.1	tracketch	17
4.2	Simulation	29
4.3	Etching	37
4.4	Physics	42
4.5	Wavefront	54
4.6	Stopping Power	57
5	Indices and tables	59
	Python Module Index	61
	Index	63

A Python toolkit for simulating track formation in CR-39 nuclear track detectors irradiation with ions.

The work is published in the article [A unified model for etched-track formation in CR-39 detectors using amorphous track structure theory](#) by Jeppe Brage Christensen.

 Note

Due to its dependency on [libamtrack](#), **tracketch** currently runs on **Linux only**. Most Windows installations support WSL installations.

GETTING STARTED

1.1 Installation

Clone the repository and install in a virtual environment:

```
git clone <repo-url> tracketch
cd tracketch
python -m venv .venv
source .venv/bin/activate
pip install -e ".[numba]"
```

This installs tracketch with the Numba-accelerated Dijkstra backend, which works on any Linux machine.

Note

tracketch depends on [libamtrack](#) and is currently **Linux only**.

Note

This release currently supports only 2D axisymmetric simulations (straight tracks).

1.1.1 Optional: C++ Dijkstra backend

For ~100x faster arrival-time computation:

```
pip install -e ".[numba,cpp]"
cd tracketch/wavefront/dijkstra/cpp
python setup_dijkstra.py build_ext --inplace
```

1.2 Quick example

```
import matplotlib.pyplot as plt
from tracketch import TrackSimulator

# 270 MeV/u carbon-12 hitting CR-39
sim = TrackSimulator(
    particle_name="12C",
    start_energy_MeV_u=270.0,
)

# Plot track shape after 1 and 3 hours of etching
fig, ax = plt.subplots()
```

(continues on next page)

(continued from previous page)

```

for t in [1.0, 3.0]:
    r, z = sim.get_iso_time_contour(etching_time_h=t)
    ax.plot(r, z, label=f"{t} h")
    ax.plot(-r, z, color=ax.lines[-1].get_color())

ax.set_xlabel("r / um")
ax.set_ylabel("z / um")
ax.invert_yaxis()
ax.legend(title="Etch time")
ax.set_aspect("equal")
plt.tight_layout()
plt.show()

```

The *TrackSimulator* automatically computes the dose map, etch-rate map, and arrival-time map on construction. See *Examples* for more detailed scripts.

1.3 Etch-rate model selection

tracketch uses one shared default etch-rate calibration model, "Doerschel_etching", for all particle species.

- The etch-rate model is **not** selected from `particle_name`.
- `particle_name` only controls stopping power / dose-map physics.
- You can provide a custom etch model explicitly when needed.

```

from tracketch import TrackSimulator, load_etchrate_model

etch_model = load_etchrate_model("my_custom_model")

sim = TrackSimulator(
    particle_name="12C",
    start_energy_MeV_u=270.0,
    etch_model=etch_model,
)

```

1.4 Particles and stopping-power sources

Particle names follow the "<A><symbol>" convention, where *A* is the mass number and *symbol* is the element symbol – for example "12C", "1H", "56Fe".

The stopping-power source is chosen with the `stopping_power_source` parameter:

"SRIM" (default)

Uses tabulated SRIM data. Only the following ten particles are supported:

```

import tracketch
print(tracketch.SRIM_PARTICLES)
# ('1H', '2H', '3H', '4He', '7Li', '9Be', '11B', '12C', '14N', '16O')

```

Requesting any other particle raises a `ValueError` with a hint to switch to "libamtrack".

Note

All three hydrogen isotopes are supported: "1H" (proton), "2H" (deuteron), "3H" (triton). The CSDA range scales linearly with mass number *A*; LET at the same MeV/u is identical across isotopes.

"libamtrack"

Uses the PSTAR/ASTAR parametrisation via [libamtrack](#). Accepts any nuclide recognised by libamtrack, including heavy ions such as "56Fe" or "238U".

```
from tracketch import TrackSimulator

# Heavy ion -- switch to libamtrack for stopping power
sim = TrackSimulator(
    particle_name="56Fe",
    start_energy_MeV_u=1000.0,
    stopping_power_source="libamtrack",
)
```

1.5 Target material

Two materials are supported:

```
import tracketch
print(tracketch.MATERIALS)    # ('CR39', 'water')
```

Pass the name as `material_name="CR39"` (default, density 1.31 g/cm³) or `material_name="water"` (density 1.00 g/cm³). An unknown name raises a `ValueError`.

1.6 Simulation grid

The simulation uses a 2-D cylindrical grid in (*r*, *z*). The radial axis is log-spaced (to resolve the narrow dose core near *r* = 0); the depth axis is uniform. Default values:

Parameter	Default	Description
<code>r_min_um</code>	1e-4 um	Inner radial boundary (start of log-spaced axis)
<code>r_max_um</code>	20 um	Outer radial boundary
<code>z_max_um</code>	40 um	Maximum depth along the ion path
<code>n_points_r</code>	400	Number of radial grid points
<code>n_points_z</code>	100	Number of depth grid points

All five parameters can be passed directly to `TrackSimulator`:

```
from tracketch import TrackSimulator

sim = TrackSimulator(
    particle_name="12C",
    start_energy_MeV_u=270.0,
    r_max_um=40,      # wider radial window
    z_max_um=80,      # deeper track
    n_points_r=600,   # finer radial resolution
    n_points_z=200,   # finer depth resolution
)
```

Increasing resolution improves accuracy at the cost of compute time. For publication-quality results consider doubling both `n_points_r` and `n_points_z`. For quick exploration the defaults are a good starting point.

Note

Enabling parallel dose-map computation with `n_jobs=-1` reduces wall time for large grids ($\geq 50\,000$ total

cells) or when using `stopping_power_source="libamtrack"`. For the default 400 x 100 grid the inter-process overhead outweighs the gain; leave `n_jobs=1` (the default) in that case.

1.7 Building the documentation

```
cd docs
make html
```

Then open `_build/html/index.html` in a browser.

EXAMPLES

The `examples/` directory contains runnable scripts that demonstrate the main features of tracketch. Each section below shows the code and explains what it does.

2.1 Simulating track shapes

The most common use-case: create a `TrackSimulator`, then extract the etched track contour at several etching durations.

```
"""
Track shapes for different ions and etching times.

Simulates the full pipeline: dose map -> etch rate -> arrival time -> track
contour. Shows how track shape evolves with etching time for a 270 MeV/u
carbon ion in CR39.

If the loaded etch model carries anchor velocity uncertainties, a 1-sigma
shaded band is added around each contour.
"""

import matplotlib.pyplot as plt
from tracketch import TrackSimulator

sim = TrackSimulator(
    particle_name="12C",
    start_energy_MeV_u=270.0,
    r_max_um=5,
)

# calculate the track contours for different etching times
etch_times_h = [0.5, 1.0, 2.0, 3.0]

fig, ax = plt.subplots()
for idx, t_h in enumerate(etch_times_h):
    color = plt.cm.viridis(idx / len(etch_times_h))

    unc_result = sim.get_iso_time_contour_with_uncertainty(t_h)
    if unc_result is not None:
        (r, z), z_band, r_lo, r_hi = unc_result
        ax.fill_betweenx(z_band, r_lo, r_hi, alpha=0.25, color=color)
        ax.fill_betweenx(z_band, -r_hi, -r_lo, alpha=0.25, color=color)
    else:
        r, z = sim.get_iso_time_contour(etching_time_h=t_h)
```

(continues on next page)

(continued from previous page)

```

ax.plot(r, z, label=f"{t_h}", color=color)
ax.plot(-r, z, color=color) # mirror for full track profile

ax.set_xlabel("r / um")
ax.set_ylabel("z / um")
ax.set_title("12C @ 270 MeV/u: track shape evolution")
ax.invert_yaxis()
ax.legend(title="Etch time / h")
ax.set_aspect("equal")

```

The contour is returned as (r, z) arrays in um. Mirroring -r gives the full axially-symmetric profile.

2.2 Hydrogen isotopes – proton, deuteron, triton

Added in version 0.1.

tracketch fully supports all three hydrogen isotopes ("1H", "2H", "3H") as particle names. The physics is correct for each isotope:

- **LET** is identical at the same kinetic energy per nucleon (MeV/u) – it depends only on charge and velocity, not on mass number.
- **CSDA range** scales linearly with mass number A, because a heavier isotope carries more momentum at the same MeV/u.
- **Total kinetic energy** (MeV) = energy per nucleon x A.

The example below computes track radii for proton, deuteron, and triton across a range of energies, illustrating the range difference at the same MeV/u:

```

"""
Plotting track radii as a function of ion energy.

This example computes the track radius in CR-39 for a grid of ion energies
and two etching times, using :class:`~tracketch.TrackSimulator`. Results
are collected in a :class:`~pandas.DataFrame` and visualised with
``seaborn.lineplot``.
"""

import numpy as np
import itertools
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
from tracketch import TrackSimulator
import tqdm

etch_times_h = [5, 10]
particles = ["1H", "2H", "3H"]
energies_MeV_u = np.logspace(-2, 2, num=300)

# Create all unique combinations
combinations = list(itertools.product(particles, energies_MeV_u))
combinations_df = pd.DataFrame(combinations, columns=["particle_name", "energy_MeV_u",
→])

result_df = pd.DataFrame(

```

(continues on next page)

(continued from previous page)

```

columns=[
    "particle_name",
    "energy_MeV_u",
    "energy_MeV",
    "CSDA_um",
    "etch_time_h",
    "radius_um",
]
)

# iterate through combinations and calculate track radius for each
for idx, row in tqdm.tqdm(combinations_df.iterrows(), total=len(combinations_df)):
    particle = row["particle_name"]
    energy_MeV_u = row["energy_MeV_u"]

    # create simulator
    sim = TrackSimulator(
        particle_name=particle,
        start_energy_MeV_u=energy_MeV_u,
    )

    for etching_time_h in etch_times_h:
        # get results for this etching time
        new_row = pd.DataFrame(
            {
                "Particle": [particle],
                "Energy (MeV/u)": [energy_MeV_u],
                "Etch time / h": [etching_time_h],
                "Energy / MeV": [sim.start_energy_MeV],
                "CSDA / um": [sim.CSDA_range_um],
                "Track radius / um": [
                    sim.get_track_radius_um(etch_time_h=etching_time_h)
                ],
                "Track length / um": [
                    sim.get_track_length_um(etch_time_h=etching_time_h)
                ],
            }
        )
        result_df = pd.concat([result_df, new_row], ignore_index=True)

# %%

fig, axes = plt.subplots(ncols=2, nrows=2, sharey=True, figsize=(10, 10))
fig.suptitle("Track radius in CR-39 for hydrogen isotopes", y=0.92)
fig.subplots_adjust(wspace=0.05)

(ax1, ax2, ax3, ax4) = axes.flatten()

sns.lineplot(
    data=result_df,
    x="CSDA / um",
    y="Track radius / um",
    hue="Particle",
    style="Etch time / h",
    ax=ax1,
)

```

(continues on next page)

(continued from previous page)

```

sns.lineplot(
    data=result_df,
    x="Energy (MeV/u)",
    y="Track radius / um",
    hue="Particle",
    style="Etch time / h",
    ax=ax2,
)

sns.lineplot(
    data=result_df,
    x="CSDA / um",
    y="Track length / um",
    hue="Particle",
    style="Etch time / h",
    ax=ax3,
)

sns.lineplot(
    data=result_df,
    x="Energy (MeV/u)",
    y="Track length / um",
    hue="Particle",
    style="Etch time / h",
    ax=ax4,
)

for ax in axes.flatten():
    ax.grid(alpha=0.3)
    ax.set_xscale("log")

plt.show()

# %%

```

2.3 Radial dose distributions

The radial dose distribution (RDD) describes how dose falls off with distance from the ion path. tracketch wraps the Cucinotta model from libamtrack and scales it to the target material.

```

"""
Radial dose distributions (RDDs) for different ions.

Plots the Cucinotta RDD for protons, helium, and carbon ions at selected
energies. The RDD describes how dose falls off radially from the ion track.
"""

import numpy as np
import matplotlib.pyplot as plt
from tracketch import get_RDD_Gy

r_m = np.logspace(-9, -3, 200)

ions = [
    ("1H", 100.0),

```

(continues on next page)

(continued from previous page)

```

    ("4He", 150.0),
    ("12C", 270.0),
]

fig, ax = plt.subplots()
for particle, E_MeV_u in ions:
    dose = get_RDD_Gy(r_m, E_MeV_u, particle, RDD_name="Cucinotta")
    ax.loglog(r_m * 1e6, dose, label=f"{particle} @ {E_MeV_u} MeV/u")

ax.set_xlabel("Radius / um")
ax.set_ylabel("Dose / Gy")
ax.legend()
ax.set_title("Radial dose distribution (Cucinotta)")
plt.tight_layout()
plt.show()

```

2.4 LET and CSDA range

Linear energy transfer (LET) and CSDA range can be queried from SRIM tables or from libamtrack. The plot includes hydrogen isotopes to illustrate that LET curves for 1H, 2H, and 3H coincide while CSDA ranges differ by a factor of A.

```

"""
LET and CSDA range for different ions in CR39.

Plots linear energy transfer (LET) and continuous slowing-down approximation
(CSDA) range as a function of kinetic energy for hydrogen isotopes, helium,
and carbon.

Hydrogen isotopes (`1H`, `2H`, `3H`) share the same LET curve because
LET depends only on charge and velocity (MeV/u), not on mass number. Their
CSDA ranges differ by a factor equal to the mass number *A* -- a deuteron
travels twice as far as a proton at the same MeV/u.
"""

import numpy as np
import matplotlib.pyplot as plt
from tracketch import get_LET_keV_um, get_CSDA_um

energies = np.logspace(-1, 2.5, 50)
particles = ["1H", "2H", "3H", "4He", "12C"]

fig, (ax_let, ax_csd) = plt.subplots(1, 2, figsize=(10, 4))

for particle in particles:
    LET = [get_LET_keV_um(E, particle, "CR39", source="SRIM") for E in energies]
    CSDA = [get_CSDA_um(E, particle, "CR39", source="SRIM") for E in energies]

    ax_let.loglog(energies, LET, label=particle)
    ax_csd.loglog(energies, CSDA, label=particle)

ax_let.set_xlabel("Energy / (MeV/u)")
ax_let.set_ylabel("LET / (keV/um)")
ax_let.set_title("Linear energy transfer in CR39")
ax_let.legend()

```

(continues on next page)

(continued from previous page)

```
ax_csda.set_xlabel("Energy / (MeV/u)")
ax_csda.set_ylabel("CSDA range / um")
ax_csda.set_title("CSDA range in CR39")
ax_csda.legend()

plt.tight_layout()
plt.show()
```

2.5 Customising the simulation grid

By default *TrackSimulator* uses a fixed radial and depth grid. For ions with very long tracks, very short tracks, or when a different resolution is needed, individual grid parameters can be passed as plain keyword arguments:

```
sim = TrackSimulator(
    particle_name="12C",
    start_energy_MeV_u=10.0,
    r_max_um=40,    # wider radial grid
    z_max_um=80,    # deeper depth grid
    n_points_z=200,
)
```

The full example below runs the same ion three times (default, wide, and coarse grids) and overlays the resulting iso-time contours:

```
"""
Customising the simulation grid.

By default, :class:`~tracketch.TrackSimulator` uses a grid that covers
r in [1e-4, 20] um and z in [0, 40] um. For ions with very long or very
short tracks, or when you need higher/lower resolution, you can override
individual grid parameters as plain keyword arguments:

* ``r_max_um`` -- maximum radial extent of the grid (um)
* ``z_max_um`` -- maximum depth (um); should exceed the CSDA range
* ``r_min_um`` -- inner radial boundary (um)
* ``n_points_r`` -- radial resolution (more points -> slower but finer RDD)
* ``n_points_z`` -- depth resolution

This example runs the same 10 MeV/u carbon ion three times:
1. Default grid
2. Wider, deeper grid (suitable for high-energy heavy ions)
3. Compact low-resolution grid (fast scan / debugging)

and overlays the three iso-time contours to show the effect.
"""

import matplotlib.pyplot as plt
from tracketch import TrackSimulator

particle = "12C"
energy_MeV_u = 10.0
etch_time_h = 5.0

configs = [
```

(continues on next page)

(continued from previous page)

```

dict(label="default", color="C0"),
dict(label="wide grid", color="C1", r_max_um=40, z_max_um=80),
dict(label="coarse grid", color="C2", n_points_r=100, n_points_z=50),
]

fig, ax = plt.subplots(figsize=(5, 6))

for cfg in configs:
    label = cfg.pop("label")
    color = cfg.pop("color")

    sim = TrackSimulator(
        particle_name=particle,
        start_energy_MeV_u=energy_MeV_u,
        **cfg,
    )

    r, z = sim.get_iso_time_contour(etch_time_h)
    ax.plot(r, z, label=label, color=color)

    # Mark the bulk-etch depth for the default run only
    if label == "default":
        z_bulk = sim.etch_model.V_bulk_um_h * etch_time_h
        ax.axhline(
            z_bulk,
            color="gray",
            linestyle="--",
            linewidth=1,
            label=f"bulk surface ({z_bulk:.1f} um)",
        )

ax.invert_yaxis()
ax.set_xlabel("r / um")
ax.set_ylabel("z / um")
ax.set_title(f"{particle} @ {energy_MeV_u} MeV/u, t = {etch_time_h} h")
ax.legend()
plt.tight_layout()
plt.show()

# %%
print(f"CSDA range: {sim.CSDA_range_um:.1f} um")
print(
    f"Grid r: [{sim._r_lims_um[0]:.1e}, {sim._r_lims_um[1]:.0f}] um "
    f"({len(sim._r_grid_um)} points)"
)
print(
    f"Grid z: [{sim._z_lims_um[0]:.0f}, {sim._z_lims_um[1]:.0f}] um "
    f"({len(sim._z_grid_um)} points)"
)

```

2.6 Comparing arrival-time solvers

tracketch ships two wavefront propagation backends: Fast Marching (FMM) and Dijkstra. This example compares them on a uniform-speed grid.

```
"""
Arrival time maps using different marching backends.

Compares FMM and Dijkstra (Numba) on a simple uniform-speed grid.
Both should produce similar results; Dijkstra supports non-uniform grids
natively while FMM requires interpolation to a uniform grid.
"""

import numpy as np
import matplotlib.pyplot as plt
from tracketch.wavefront import arrival_time_fmm, arrival_time_dijkstra_fast

# Simple grid with uniform etch rate
r_um = np.linspace(0.01, 5.0, 80)
z_um = np.linspace(0.0, 10.0, 150)
speed = np.ones((len(z_um), len(r_um)))

t_fmm = arrival_time_fmm(r_um, z_um, speed, r_is_logscaled=False)
t_dij = arrival_time_dijkstra_fast(r_um, z_um, speed, r_is_logscaled=False)

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(10, 4), sharey=True)

for ax, t_map, title in [(ax1, t_fmm, "FMM"), (ax2, t_dij, "Dijkstra (Numba)"]]:
    im = ax.pcolormesh(r_um, z_um, t_map, shading="auto")
    ax.set_xlabel("r / um")
    ax.set_title(title)
    fig.colorbar(im, ax=ax, label="Arrival time / hr")

ax1.set_ylabel("z / um")
ax1.invert_yaxis()
plt.tight_layout()
plt.show()
```

ALGORITHMS

This page gives a brief overview of the computational methods used inside tracketch. For full API details see [API Reference](#).

3.1 Simulation pipeline

A *TrackSimulator* runs three stages:

1. **Dose map** – For each depth step along the ion path, the *radial dose distribution* (RDD) is evaluated using a model from libamtrack (e.g. Cucinotta). SRIM stopping-power tables provide LET and residual energy at each depth, so the RDD changes as the ion slows down.
2. **Etch-rate map** – The calibrated *EtchRateModel* converts local dose D to a local etch velocity $V(D)$ via monotonic PCHIP interpolation in log-dose space. Undamaged material etches at the constant bulk rate V_{bulk} .
3. **Arrival-time map** – Starting from the detector surface ($z = 0$), the etchant front is propagated through the spatially varying speed field. This is a shortest-path / wavefront problem solved by one of the backends below.

3.2 Arrival-time solvers

The etch-rate map defines a *speed* at every grid point. The arrival time at each cell is the minimum travel time from the surface through any path, weighted by inverse speed:

$$T(\mathbf{x}) = \min_{\text{paths}} \int_0^{\mathbf{x}} \frac{ds}{V(s)}$$

tracketch provides two solver families.

3.2.1 Dijkstra (recommended)

The 2-D grid is treated as a weighted graph. Each cell is a node connected to its neighbours; edge weights are the physical travel time across the cell boundary:

$$w_{ij} = \frac{|\mathbf{x}_i - \mathbf{x}_j|}{\frac{1}{2}(V_i + V_j)}$$

Because the radial grid is log-spaced (necessary to resolve narrow RDDs at small r), cell sizes vary by orders of magnitude. Dijkstra handles non-uniform spacing natively – no regridding required.

Connectivity. Higher connectivity reduces *metrication error* (the angular bias from a discrete grid):

- **8-connected** (default) – king’s moves; ~2 % error; fastest.
- **16-connected** – adds knight’s moves (+/-2, +/-1); ~1 % error.
- **32-connected** – extended knight’s moves; ~0.5 % error.

Backends. The graph is built with Numba JIT and then solved with `scipy.sparse.csgraph.dijkstra`:

- `dijkstra_numba` – pure Python + Numba; always available.
- `dijkstra_cpp` – C++/pybind11 implementation of the graph builder; ~50-100x faster. Requires compilation (see [Getting Started](#)).

3.2.2 Fast Marching Method (FMM)

The `scikit-fmm` library solves the Eikonal equation on a *uniform* grid. Since the native radial grid is log-spaced, tracketch interpolates the speed map onto a uniform grid, runs FMM, then interpolates back.

FMM supports **tilted tracks** (`theta_deg > 0`), where the wavefront starts along a diagonal rather than a flat surface. Dijkstra does not yet support tilting.

3.3 Etch-rate model

`EtchRateModel` represents $V(D)$ as a monotonic PCHIP (Piecewise Cubic Hermite Interpolating Polynomial) spline through a set of dose-velocity anchor points. Key design choices:

- Anchor velocities are stored as *cumulative increments* to enforce monotonicity by construction.
- A synthetic low-dose anchor at 10^{-3} Gy is prepended so that the spline smoothly converges to V_{bulk} at low doses.
- Extrapolation beyond the highest anchor uses the last-segment slope (PCHIP mode) or clamps to the last value.

The model can be serialised to JSON and shared across simulations.

API REFERENCE

4.1 tracketch

tracketch – simulation of ion tracks in CR-39 nuclear track detectors.

```
class tracketch.TrackSimulator(particle_name: str, start_energy_MeV_u: float, etch_model:  
                               EtchRateModel | None = None, RDD_name: str = 'Cucinotta',  
                               arrival_time_method_name: str = 'dijkstra_cpp', dijkstra_connectivity:  
                               int = 16, rz_lims_dict: dict | None = None, r_min_um: float | None =  
                               None, r_max_um: float | None = None, z_max_um: float | None = None,  
                               n_points_r: int | None = None, n_points_z: int | None = None,  
                               material_name: str = 'CR39', stopping_power_source: str = 'SRIM',  
                               n_straggling_sigma: int = 1, n_uniform_multiplier: int = 3, n_jobs: int  
                               = 1, log_level: int | str | None = None, normalise_to_LET: bool = False,  
                               logscale_r: bool = True, theta_deg: float = 0.0)
```

Bases: `object`

Simulate chemical etching of a single ion track in CR-39.

The simulation proceeds in three stages:

1. **Dose map** – radial dose distribution (RDD) integrated along the ion path, accounting for energy loss and optional range straggling.
2. **Etch-rate map** – the calibrated `etch_model` converts local dose to local etch velocity, with optional debris-damping correction.
3. **Arrival-time map** – shortest-time wavefront propagation (Dijkstra or Fast Marching) from the detector surface into the bulk.

From the arrival-time map, iso-time contours, track radii, and track depths are extracted for any desired etching duration.

Parameters

- **particle_name** (*str*) – Ion species identifier in the form "<A><symbol>", e.g. "12C", "1H", "56Fe", "238U". When `stopping_power_source="SRIM"` (default) the particle must be one of `tracketch.SRIM_PARTICLES`; switch to `stopping_power_source="libamtrack"` to use any nuclide recognised by libamtrack.
- **start_energy_MeV_u** (*float*) – Kinetic energy at the detector surface in MeV per nucleon.
- **etch_model** (*tracketch.etching.etch_rate_model.EtchRateModel or None*) – Calibrated dose -> etch-rate model. If *None*, a default model is loaded ("Doerschel_etching").
- **RDD_name** (*str*) – Radial dose distribution model name (default "Cucinotta").

- **arrival_time_method_name** (*str*) – Wavefront algorithm: "dijkstra" (default, auto-selects fastest available backend), "dijkstra_cpp", "dijkstra_numba", or "fmm".
- **dijkstra_connectivity** (*int*) – Neighbour connectivity for Dijkstra: 8, 16, or 32.
- **r_min_um** (*float* or *None*) – Minimum radial coordinate in um (default 1e-4).
- **r_max_um** (*float* or *None*) – Maximum radial coordinate in um (default 20).
- **z_max_um** (*float* or *None*) – Maximum depth coordinate in um (default 40).
- **n_points_r** (*int* or *None*) – Number of radial grid points (default 400).
- **n_points_z** (*int* or *None*) – Number of depth grid points (default 100).
- **rz_lims_dict** (*dict* or *None*) – Legacy grid specification dict. Individual keyword arguments above take precedence over keys in this dict when both are supplied.
- **material_name** (*str*) – Target material – "CR39" (default) or "water".
- **stopping_power_source** (*str*) – Stopping-power database: "SRIM" (default, tabulated data for common ions) or "libamtrack" (supports any nuclide, uses the PSTAR/ASTAR parametrisation).
- **n_straggling_sigma** (*int*) – Number of longitudinal-straggling sigma to include (0 = none).
- **n_uniform_multiplier** (*int*) – Resolution multiplier when regridding to uniform spacing for FMM.
- **n_jobs** (*int*) – Number of parallel workers for the RDD/dose-map computation. -1 uses all available CPU cores (via `joblib`). Parallel execution benefits large grids ($n_points_r \times n_points_z > \sim 50\,000$) or `stopping_power_source="libamtrack"`. For the default grid the inter-process overhead outweighs the gain; keep `n_jobs=1` (sequential) unless you have increased the grid resolution. Default 1.
- **log_level** (*int* or *str* or *None*) – Set the logging level for the tracketch logger, e.g. `logging.DEBUG`, `logging.INFO`, or the string "DEBUG". When *None* (default) the logger is left untouched.

dose_map

Local dose in Gy.

Type

ndarray, shape (n_z, n_r)

etch_rate_map

Local etch velocity in um/hr.

Type

ndarray, shape (n_z, n_r)

arrival_time_map

Shortest etchant arrival time in hours.

Type

ndarray, shape (n_z, n_r)

CSDA_range_um

Continuous-slowing-down-approximation range in um.

Type

float

energy_profile_MeV_u

Kinetic energy vs. depth (with straggling).

Type

ndarray

LET_profile_keV_um

Linear energy transfer vs. depth (with straggling).

Type

ndarray

etch_model

The active dose -> etch-rate model.

Type*tracketch.etching.etch_rate_model.EtchRateModel***Notes****Track radius and depth**

After construction, extract the observable track geometry for any etching duration with *get_track_radius_um()* and *get_track_length_um()*:

```
sim = TrackSimulator(particle_name="12C", start_energy_MeV_u=270.0)

radius_um = sim.get_track_radius_um(etch_time_h=3.0)
depth_um = sim.get_track_length_um(etch_time_h=3.0) # pit depth below surface
```

Both return float("nan") when the track is not yet open (too short an etching time) or when the ion stops before reaching the detector surface.

Track contour

For the full 2-D pit shape use *get_iso_time_contour()*, which returns (r, z) arrays in um:

```
r, z = sim.get_iso_time_contour(etching_time_h=3.0)

import matplotlib.pyplot as plt
fig, ax = plt.subplots()
ax.plot(r, z)
ax.plot(-r, z) # mirror for full axisymmetric view
ax.invert_yaxis()
ax.set_aspect("equal")
```

Uncertainties

Uncertainties are propagated from the calibrated V(D) anchor-velocity uncertainties. These must first be estimated from a calibration dataset using *calibration.lib_optimiser.estimate_parameter_uncertainties()* and then stored on the etch model:

```
from calibration.lib_optimiser import estimate_parameter_uncertainties

result = estimate_parameter_uncertainties(
    etch_model=sim.etch_model,
    models_dict=my_calibration_data, # same dict used during fitting
)
sim.etch_model.anchor_velocity_uncertainties_um_h = (
    result["anchor_velocity_uncertainties_um_h"]
)
sim.etch_model.V_max_uncertainty_um_h = result.get("V_max_uncertainty_um_h")
```

Once the uncertainties are attached, use the *_with_uncertainty* variants:

```

radius_um, radius_unc_um = sim.get_track_radius_um_with_uncertainty(
    etch_time_h=3.0, # etching duration
    n_sigma=1.0,     # propagate +/-1 sigma of V(D) anchors
)

result = sim.get_track_length_um_with_uncertainty(
    etch_time_h=3.0,
    n_sigma=1.0,
)
if result is not None:
    depth_um, depth_unc_um = result

print(f"radius = {radius_um:.2f} +/- {radius_unc_um:.2f} um")
print(f"depth  = {depth_um:.2f} +/- {depth_unc_um:.2f} um")

```

Both methods return nan uncertainties when a bound contour cannot be extracted. `get_track_radius_um_with_uncertainty()` also floors the uncertainty at half the minimum radial grid step, since the contour extraction cannot resolve sub-cell differences.

For a full uncertainty *band* around the track contour see `get_iso_time_contour_with_uncertainty()`.

calculate_RDD_Gy(*E_MeV_u*: *float*) → *ndarray*

Calculate the radial dose distribution at a given energy.

Parameters

E_MeV_u (*float*) – Kinetic energy in MeV per nucleon.

Returns

Dose in Gy at each radial grid point.

Return type

ndarray

get_iso_time_contour(*etching_time_h*: *float*) → *tuple*[*ndarray*, *ndarray*]

Extract the iso-time contour from the arrival-time map.

Parameters

etching_time_h (*float*) – Etching duration in hours.

Returns

r_coords, **z_coords** – Coordinates along the longest contour segment.

Return type

tuple[*ndarray*, *ndarray*]

get_iso_time_contour_with_uncertainty(*etching_time_h*: *float*, *n_sigma*: *float* = 1.0, *n_band_points*: *int* = 200) → *tuple*[*tuple*[*ndarray*, *ndarray*], *ndarray*, *ndarray*, *ndarray*] | *None*

Iso-time contour with uncertainty band from V(D) anchor uncertainties.

Computes arrival-time maps for V(D) $\pm$ *n_sigma**dV, extracts the three contours, and interpolates the upper/lower bounds onto a common z grid suitable for `ax.fill_betweenx`.

Parameters

- **etching_time_h** (*float*) – Etching duration in hours.
- **n_sigma** (*float*) – Number of standard deviations to propagate (default 1).
- **n_band_points** (*int*) – Number of points on the common z grid for the band (default 200).

Returns

- tuple of ((r_central, z_central), z_band, r_lo, r_hi) or *None*.
- Returns **None* (and logs a warning) when the etch model carries no*
- *anchor velocity uncertainties*. Assign the output of
- `estimate_parameter_uncertainties` to
- `etch_model.anchor_velocity_uncertainties_um_h` first.

get_track_detectability(etch_time_h: *float*, min_radius_um: *float* = 0.5, min_depth_um: *float* = 0.3, min_cone_half_angle_deg: *float* = 2.0, threshold_percent: *float* = 5.0) → *dict*

Geometric detectability metrics for a given etching time.

A track is considered detectable when **all three** criteria are met:

1. `radius_um >= min_radius_um`
2. `depth_um >= min_depth_um` (pit depth below bulk surface)
3. `cone_half_angle_deg >= min_cone_half_angle_deg`

Returns

Keys: `detected`, `radius_um`, `depth_um`, `cone_half_angle_deg`, `radius_ok`, `depth_ok`, `angle_ok`.

Return type

dict

get_track_length_um(etch_time_h: *float* | *list*, relative_to_surface: *bool* = *True*) → *float* | *ndarray*

Track depth at `r = 0` for a given etching time.

Parameters

- **etch_time_h** (*float* or *list*[*float*]) – Etching duration(s) in hours.
- **relative_to_surface** (*bool*) – If *True* (default), subtract the bulk-etched surface position so the result is the pit depth below the current surface.

Returns

Depth in `um`, or `nan` if the track does not reach `r = 0`.

Return type

float or *ndarray*

get_track_length_um_with_uncertainty(etch_time_h: *float*, relative_to_surface: *bool* = *True*, n_sigma: *float* = 1.0) → *tuple*[*float*, *float*] | *None*

Track depth with uncertainty propagated from `V(D)` anchor uncertainties.

Evaluates the arrival-time map for `V(D)+n_sigma*dV` and `V(D)-n_sigma*dV`, reads the depth at `r = 0` for each, and returns the central depth together with half the spread as the uncertainty estimate.

Parameters

- **etch_time_h** (*float*) – Etching duration in hours.
- **relative_to_surface** (*bool*) – If *True* (default), subtract the bulk-etched surface depth.
- **n_sigma** (*float*) – Number of standard deviations to propagate (default 1).

Returns

- tuple of (`depth_um`, `depth_uncertainty_um`) or *None*.
- Returns **None* (and logs a warning) when the etch model carries no*
- *anchor velocity uncertainties*.

get_track_radius_um(*etch_time_h*: *float*, *threshold_percent*: *float* = 5.0) → *float*

Track opening radius from iso-time contour deviation.

Parameters

- **etch_time_h** (*float*) – Etching duration in hours.
- **threshold_percent** (*float*) – Deviation threshold as percentage of bulk depth (default 5 %).

Returns

Track radius in um, or nan if no contour is found.

Return type

float

get_track_radius_um_with_uncertainty(*etch_time_h*: *float*, *threshold_percent*: *float* = 5.0, *n_sigma*: *float* = 1.0) → *tuple*[*float*, *float*]

Track radius with uncertainty propagated from V(D) anchor uncertainties.

Evaluates the etch-rate and arrival-time maps for $V(D)+n_sigma*dV$ and $V(D)-n_sigma*dV$, extracts a radius from each, and returns the central radius together with half the spread as the uncertainty estimate.

Parameters

- **etch_time_h** (*float*) – Etching duration in hours.
- **threshold_percent** (*float*) – Deviation threshold as percentage of bulk depth (default 5 %).
- **n_sigma** (*float*) – Number of standard deviations to propagate (default 1).

Returns

- **radius_um** (*float*) – Central track radius in um.
- **radius_uncertainty_um** (*float*) – Half-spread of the upper/lower bound radii (1-sigma estimate).

Raises

ValueError – If the etch model carries no anchor velocity uncertainties.

is_track_detected(*etch_time_h*: *float*, *min_radius_um*: *float* = 0.5, *min_depth_um*: *float* = 0.3, *min_cone_half_angle_deg*: *float* = 2.0, *threshold_percent*: *float* = 5.0) → *bool*

Return *True* if the track is detectable at this etching time.

Convenience wrapper around `get_track_detectability()`.

plot_LET_energy_profiles() → *tuple*

Plot energy and LET profiles vs. depth.

plot_iso_time_contour(*ax*: *Axes*, *etching_time_h*: *float*) → *tuple*[*ndarray*, *ndarray*]

Plot the iso-time contour on *ax*.

Returns

r_coords, *z_coords*

Return type

tuple[*ndarray*, *ndarray*]

plot_map(*name*: *str* = 'dose', *grayscale_mode*: *bool* = False, *plot_contours*: *bool* = True, *annotate_figure*: *bool* = True) → *tuple*

Plot the dose, etch-rate, or arrival-time map.

plot_track_radius(*etch_time_h*: *float*, *ax*: *Axes* | *None* = None, *show_contour*: *bool* = True, *threshold_percent*: *float* = 5.0) → *tuple*

Plot iso-time contour with track-radius indicators.

Return type

fig, ax

recalculate_from_current_etch_model() → *None*

Recompute maps using the current etch model.

Alias for `update_etch_model_and_recalculate(None)`.**update_etch_model_and_recalculate**(*new_etch_model*: *EtchRateModel* | *None* = *None*) → *None*

Replace the etch model and recompute etch-rate & arrival-time maps.

The dose map is *not* recomputed (it depends only on physics, not the etch model).**Parameters****new_etch_model** (*tracketch.etching.etch_rate_model.EtchRateModel* or *None*) – New model. If *None*, recomputes using the current model (useful after in-place parameter changes).

```
class tracketch.EtchRateModel(anchor_doses_Gy: ndarray | list[float] = array([1.00000000e+01,
6.30957344e+02, 3.98107171e+04, 2.51188643e+06, 1.58489319e+08,
1.00000000e+10]), V_bulk_um_h: float = 1.73, V_max_um_h: float |
None = None, V_max_uncertainty_um_h: float | None = None,
anchor_velocities_um_h: ndarray | list[float] | None = None,
anchor_velocity_uncertainties_um_h: ndarray | list[float] | None = None,
name: str | None = None, extrapolation_mode: str = 'pchip',
debris_alpha: float | None = None, debris_beta: float = 1.0)
```

Bases: *object***LOW_DOSE_ANCHOR_GY**: *float* = **0.001****property debris_damping_enabled**: *bool*

Whether debris damping is active.

eval(*dose_Gy*: *float* | *ndarray*) → *float* | *ndarray*

Evaluate etch rate at given dose(s).

Parameters**dose_Gy** (*float* or *array-like*) – Dose in Gy.**Returns****V_um_h** – Etch rate in um/hour Returns scalar if input is scalar, array if input is array
Returns **V_bulk_um_h** for NaN inputs Always $\geq$ **V_bulk_um_h** (physically consistent)**Return type***float* or *ndarray***eval_uncertainty_band**(*dose_Gy*: *ndarray*, *n_sigma*: *float* = 1.0, *max_sigma_log10*: *float* = 0.5) →
tuple[*ndarray*, *ndarray*] | *None*

Return lower and upper 1-sigma etch-rate bands at the given doses.

Converts per-anchor velocity uncertainties to log10-velocity space, interpolates them onto *dose_Gy* with PCHIP (in log-dose space), and applies the band symmetrically around `eval(dose_Gy)` in log10 space. Results are clipped to [**V_bulk**, **V_max**].**Parameters**

- **dose_Gy** (*array-like*) – Dose values at which to evaluate the band.
- **n_sigma** (*float*) – Number of standard deviations for the band width (default 1).
- **max_sigma_log10** (*float*) – Cap on interpolated sigma in log10-velocity units (default 0.5, i.e. ~factor-of-3 per sigma) to suppress artefacts from poorly-constrained high-dose anchors.

Returns

- *tuple of (lower, upper) ndarrays, or None if no anchor uncertainties*
- *are set on the model.*

get_increments() → *ndarray*

Get relative velocity increments (for optimization).

Returns

increments – Relative increments [um/hour], all ≥ 0

Return type

np.ndarray

get_log_doses() → *ndarray*

Get log10 of dose anchors (for optimization in log-space).

Returns

log_doses – log10 of anchor doses [dimensionless]

Return type

np.ndarray

get_log_velocity_increments() → *ndarray*

Get monotonic increments in log10(velocity) space.

Defines:

$u_i = \log_{10}(v_i)$ $\Delta u_0 = u_0 - \log_{10}(V_{\text{bulk_um_h}})$ $\Delta u_i = u_i - u_{i-1}$ for $i \geq 1$

Since anchor velocities are monotonic and $\geq V_{\text{bulk_um_h}}$, all returned increments are guaranteed to be ≥ 0 .

Returns

Non-negative increments in log10(velocity) space.

Return type

np.ndarray

is_monotonic(num_points: int = 256) → *bool*

Check monotonic non-decreasing behavior of $v(d)$ on a dense log-dose grid.

static linear_increments_to_log(increments: ndarray | list[float], epsilon: float = 1e-06) → *ndarray*

Convert linear increments to log-space (for unconstrained optimization).

Inverse of log_increments_to_linear.

Parameters

- **increments** (*array-like*) – Linear increments (must all be $> -\epsilon$)
- **epsilon** (*float*, *optional*) – Small offset used during forward conversion (default: $1e-6$)

Returns

log_increments – Log-transformed increments

Return type

np.ndarray

static linear_log_velocity_increments_to_optimization(log_velocity_increments: ndarray | list[float], epsilon: float = 1e-12) → *ndarray*

Map positive log-velocity increments to unconstrained optimizer parameters.

classmethod load_from_json(filepath: str, verbose: bool = False) → *EtchRateModel*

Load an etch rate model from a JSON file.

Parameters**filepath** (*str*) – Path to the JSON file**Returns**

The loaded model

Return type*EtchRateModel***Examples**

```
>>> model = EtchRateModel.load_from_json("my_model.json")
```

```
static log_increments_to_linear(log_increments: ndarray | list[float], epsilon: float = 1e-06) → ndarray
```

Convert log-space increments to linear (for unconstrained optimization).

For optimization, parameterize as: `log_increments = log(increments + epsilon)` This ensures increments are always positive without constraints.**Parameters**

- **log_increments** (*array-like*) – Log-transformed increments (can be any real number)
- **epsilon** (*float*, *optional*) – Small offset to ensure positivity (default: 1e-6)

Returns**increments** – Linear increments, all > epsilon**Return type**`np.ndarray`

```
static log_velocity_increments_to_linear(optimization_parameters: ndarray | list[float], epsilon: float = 1e-12) → ndarray
```

Map unconstrained optimizer parameters to positive log-velocity increments.

```
plot(dose_range: tuple[float, float] = (1e-06, 1000000000000.0), n_points: int = 500, show_anchors: bool = True, show_v_bulk: bool = True, show_uncertainty_band: bool = True) → tuple[Figure, Axes]
```

Plot the etch rate function.

Parameters

- **dose_range** (*tuple*, *optional*) – (min_dose, max_dose) in Gy for x-axis
- **n_points** (*int*, *optional*) – Number of evaluation points
- **show_anchors** (*bool*, *optional*) – Whether to show anchor points
- **show_v_bulk** (*bool*, *optional*) – Whether to show `V_bulk_um_h` reference line
- **show_uncertainty_band** (*bool*, *optional*) – Whether to show uncertainty band if anchor uncertainties are available

Returns**fig, ax****Return type**`matplotlib figure and axes`

```
save_to_json(filepath: str) → None
```

Save the etch rate model to a JSON file.

Parameters**filepath** (*str*) – Path to the output JSON file

Examples

```
>>> model.save_to_json("my_model.json")
```

summary() → `str`

Return a compact human-readable summary of model settings and ranges.

update_debris_params(*alpha*: `float` | `None`, *beta*: `float` | `None` = `None`) → `None`

Update debris-damping parameters.

Parameters

- **alpha** (`float` or `None`) – Characteristic aspect-ratio scale. `None` disables damping.
- **beta** (`float` or `None`) – Transition steepness. `None` keeps current value.

update_dose_anchors(*anchor_doses_Gy*: `ndarray` | `list[float]`) → `None`

Update dose anchor positions and recreate spline.

Validates that doses are strictly increasing.

Parameters

anchor_doses_Gy (*array-like*) – New dose anchor points [Gy]. Must be strictly increasing.

Raises

ValueError – If doses not strictly increasing or length doesn't match velocities

update_from_increments(*increments_um_h*: `ndarray` | `list[float]`) → `None`

Update velocities from relative increments (for optimization).

This is the recommended approach for optimization: optimize the increments directly to guarantee monotonicity by construction.

Parameters

increments_um_h (*array-like*) – Relative increments [um/hour]. Must be all ≥ 0 and have same length as anchor points.

Raises

ValueError – If increments are negative or length doesn't match If resulting velocities would be $< V_{\text{bulk_um_h}}$

update_from_log_velocity_increments(*log_velocity_increments*: `ndarray` | `list[float]`) → `None`

Update anchor velocities from non-negative $\log_{10}(\text{velocity})$ increments.

This parameterization is robust for wide dynamic ranges because optimization steps act multiplicatively on velocity.

Parameters

log_velocity_increments (*array-like*) – Increments in $\log_{10}(\text{velocity})$ space. Must all be ≥ 0 and have the same length as anchor points.

update_velocities(*new_velocities_um_h*: `ndarray` | `list[float]`) → `None`

Update anchor velocities and recreate spline.

Accepts absolute velocities, validates them for monotonicity, and converts to relative increments internally.

Useful during optimization loops.

Parameters

new_velocities_um_h (*array-like*) – New absolute velocity values for anchor points [um/hour]

Raises

ValueError – If new velocities violate monotonicity

```

V_bulk_um_h: float
V_max_um_h: float
V_max_uncertainty_um_h: float | None
name: str | None
extrapolation_mode: str
debris_alpha: float | None
debris_beta: float
anchor_velocities_um_h: ndarray
anchor_velocity_uncertainties_um_h: ndarray | None

```

`tracketch.load_etchrate_model(model_name: str) → EtchRateModel`

Load a model using the configured path.

Parameters

model_name (*str*) – The model identifier (e.g., ‘Doerschel_etching’)

Returns

The loaded model

Return type

tracketch.etching.etch_rate_model.EtchRateModel

Examples

```
>>> model = load_etchrate_model('Doerschel_etching')
```

`tracketch.save_etchrate_model(model: EtchRateModel, model_name: str) → None`

Save a model using the configured path.

Parameters

- **model** (*tracketch.etching.etch_rate_model.EtchRateModel*) – The model to save
- **model_name** (*str*) – The model identifier (e.g., ‘Doerschel_etching’)

Examples

```
>>> save_etchrate_model(my_model, 'my_custom_model')
```

`tracketch.get_RDD_Gy(r_m: ndarray, E_MeV_u: float, particle_name: str, RDD_name: str = 'Cucinotta', material_name: str = 'CR39', core_nm: float | None = None, normalise_to_LET: bool | None = None, LET_source: str = 'SRIM', verbose: bool = False) → ndarray`

Radial dose distribution scaled to *material_name*.

The RDD is computed for water (via libamtrack) and then corrected:

1. Radii are density-scaled to account for different electron ranges.
2. A dose correction restores the radial integral after the range shift.
3. A stopping-power ratio (SPR) converts dose-to-water into dose-to- target-material.
4. Optionally, the profile is renormalised so that its radial integral equals the target-material LET.

Parameters

- **r_m** (*ndarray*) – Radial distances in metres.

- **E_MeV_u** (*float*) – Kinetic energy in MeV/u.
- **particle_name** (*str*) – Ion species.
- **RDD_name** (*str*) – RDD model name (default "Cucinotta").
- **material_name** (*str*) – Target material (default "CR39").
- **core_nm** (*float* or *None*) – Core radius override in nm.
- **normalise_to_LET** (*bool* or *None*) – Force LET normalisation. *None* uses the model default.
- **LET_source** (*str*) – Stopping-power source for the SPR (default "SRIM").
- **verbose** (*bool*) – Print diagnostic information.

Returns

Dose in Gy at each radius.

Return type

ndarray

```
tracketch.get_LET_keV_um(energy_MeV_u: float | ndarray, particle_name: str, material_name: str, source: str = 'SRIM') → float | ndarray
```

Return LET in keV/um from the chosen stopping-power database.

Parameters

- **energy_MeV_u** (*float* or *ndarray*) – Kinetic energy in MeV per nucleon.
- **particle_name** (*str*) – Ion species, e.g. "12C".
- **material_name** (*str*) – Target material, e.g. "CR39".
- **source** (*str*) – "SRIM" (default) or "libamtrack".

Returns

LET in keV/um.

Return type

float or ndarray

Raises

ValueError – If *source* or *material_name* are not recognised.

```
tracketch.get_CSDA_um(energy_MeV_u: float, particle_name: str, material_name: str, source: str = 'SRIM') → float
```

Return the CSDA range in um from the chosen stopping-power database.

Parameters

- **energy_MeV_u** (*float*) – Kinetic energy in MeV/u.
- **particle_name** (*str*) – Ion species.
- **material_name** (*str*) – Target material.
- **source** (*str*) – "SRIM" (default) or "libamtrack".

Returns

CSDA range in um.

Return type

float

Raises

ValueError – If *source* or *material_name* are not recognised.

`tracketch.convert_MeV_to_MeV_u(Energy_MeV: float, particle_name: str) → float`

Convert total kinetic energy (MeV) to energy per nucleon (MeV/u).

Parameters

- **Energy_MeV** (*float*) – Total kinetic energy in MeV.
- **particle_name** (*str*) – Ion species identifier.

Returns

Energy in MeV/u.

Return type

float

`tracketch.convert_MeV_u_to_MeV(Energy_MeV_u: float, particle_name: str) → float`

Convert energy per nucleon (MeV/u) to total kinetic energy (MeV).

Parameters

- **Energy_MeV_u** (*float*) – Energy in MeV per nucleon.
- **particle_name** (*str*) – Ion species identifier.

Returns

Total kinetic energy in MeV.

Return type

float

4.2 Simulation

Track etching simulator for CR-39 nuclear track detectors.

This module provides *TrackSimulator*, which computes the 2-D dose map, etch-rate map, and etchant arrival-time map for a single ion track in CR-39, then extracts observable track geometry (radius, depth, contour).

```
class tracketch.simulation.simulator.TrackSimulator(particle_name: str, start_energy_MeV_u:
    float, etch_model: EtchRateModel | None =
    None, RDD_name: str = 'Cucinotta',
    arrival_time_method_name: str =
    'dijkstra_cpp', dijkstra_connectivity: int =
    16, rz_lims_dict: dict | None = None,
    r_min_um: float | None = None, r_max_um:
    float | None = None, z_max_um: float | None
    = None, n_points_r: int | None = None,
    n_points_z: int | None = None,
    material_name: str = 'CR39',
    stopping_power_source: str = 'SRIM',
    n_stragglingsigma: int = 1,
    n_uniform_multiplier: int = 3, n_jobs: int =
    1, log_level: int | str | None = None,
    normalise_to_LET: bool = False,
    logscale_r: bool = True, theta_deg: float =
    0.0)
```

Bases: *object*

Simulate chemical etching of a single ion track in CR-39.

The simulation proceeds in three stages:

1. **Dose map** – radial dose distribution (RDD) integrated along the ion path, accounting for energy loss and optional range straggling.

2. **Etch-rate map** – the calibrated `etch_model` converts local dose to local etch velocity, with optional debris-damping correction.
3. **Arrival-time map** – shortest-time wavefront propagation (Dijkstra or Fast Marching) from the detector surface into the bulk.

From the arrival-time map, iso-time contours, track radii, and track depths are extracted for any desired etching duration.

Parameters

- **particle_name** (*str*) – Ion species identifier in the form "<A><symbol>", e.g. "12C", "1H", "56Fe", "238U". When `stopping_power_source="SRIM"` (default) the particle must be one of `tracketch.SRIM_PARTICLES`; switch to `stopping_power_source="libamtrack"` to use any nuclide recognised by libamtrack.
- **start_energy_MeV_u** (*float*) – Kinetic energy at the detector surface in MeV per nucleon.
- **etch_model** (`tracketch.etching.etch_rate_model.EtchRateModel` or *None*) – Calibrated dose -> etch-rate model. If *None*, a default model is loaded ("`Doerschel_etching`").
- **RDD_name** (*str*) – Radial dose distribution model name (default "`Cucinotta`").
- **arrival_time_method_name** (*str*) – Wavefront algorithm: "`dijkstra`" (default, auto-selects fastest available backend), "`dijkstra_cpp`", "`dijkstra_numba`", or "`fmm`".
- **dijkstra_connectivity** (*int*) – Neighbour connectivity for Dijkstra: 8, 16, or 32.
- **r_min_um** (*float* or *None*) – Minimum radial coordinate in um (default 1e-4).
- **r_max_um** (*float* or *None*) – Maximum radial coordinate in um (default 20).
- **z_max_um** (*float* or *None*) – Maximum depth coordinate in um (default 40).
- **n_points_r** (*int* or *None*) – Number of radial grid points (default 400).
- **n_points_z** (*int* or *None*) – Number of depth grid points (default 100).
- **rz_lims_dict** (*dict* or *None*) – Legacy grid specification dict. Individual keyword arguments above take precedence over keys in this dict when both are supplied.
- **material_name** (*str*) – Target material – "`CR39`" (default) or "`water`".
- **stopping_power_source** (*str*) – Stopping-power database: "`SRIM`" (default, tabulated data for common ions) or "`libamtrack`" (supports any nuclide, uses the PSTAR/ASTAR parametrisation).
- **n_straggling_sigma** (*int*) – Number of longitudinal-straggling sigma to include (0 = none).
- **n_uniform_multiplier** (*int*) – Resolution multiplier when regriding to uniform spacing for FMM.
- **n_jobs** (*int*) – Number of parallel workers for the RDD/dose-map computation. -1 uses all available CPU cores (via `joblib`). Parallel execution benefits large grids (`n_points_r` x `n_points_z` > ~50 000) or `stopping_power_source="libamtrack"`. For the default grid the inter-process overhead outweighs the gain; keep `n_jobs=1` (sequential) unless you have increased the grid resolution. Default 1.
- **log_level** (*int* or *str* or *None*) – Set the logging level for the `tracketch` logger, e.g. `logging.DEBUG`, `logging.INFO`, or the string "`DEBUG`". When *None* (default) the logger is left untouched.

dose_map

Local dose in Gy.

Type

ndarray, shape (n_z, n_r)

etch_rate_map

Local etch velocity in um/hr.

Type

ndarray, shape (n_z, n_r)

arrival_time_map

Shortest etchant arrival time in hours.

Type

ndarray, shape (n_z, n_r)

CSDA_range_um

Continuous-slowing-down-approximation range in um.

Type

float

energy_profile_MeV_u

Kinetic energy vs. depth (with straggling).

Type

ndarray

LET_profile_keV_um

Linear energy transfer vs. depth (with straggling).

Type

ndarray

etch_model

The active dose -> etch-rate model.

Type

tracketch.etching.etch_rate_model.EtchRateModel

Notes**Track radius and depth**

After construction, extract the observable track geometry for any etching duration with *get_track_radius_um()* and *get_track_length_um()*:

```
sim = TrackSimulator(particle_name="12C", start_energy_MeV_u=270.0)

radius_um = sim.get_track_radius_um(etch_time_h=3.0)
depth_um = sim.get_track_length_um(etch_time_h=3.0) # pit depth below surface
```

Both return float("nan") when the track is not yet open (too short an etching time) or when the ion stops before reaching the detector surface.

Track contour

For the full 2-D pit shape use *get_iso_time_contour()*, which returns (r, z) arrays in um:

```
r, z = sim.get_iso_time_contour(etching_time_h=3.0)

import matplotlib.pyplot as plt
```

(continues on next page)

(continued from previous page)

```
fig, ax = plt.subplots()
ax.plot(r, z)
ax.plot(-r, z)  # mirror for full axisymmetric view
ax.invert_yaxis()
ax.set_aspect("equal")
```

Uncertainties

Uncertainties are propagated from the calibrated V(D) anchor-velocity uncertainties. These must first be estimated from a calibration dataset using `calibration.lib_optimiser.estimate_parameter_uncertainties()` and then stored on the etch model:

```
from calibration.lib_optimiser import estimate_parameter_uncertainties

result = estimate_parameter_uncertainties(
    etch_model=sim.etch_model,
    models_dict=my_calibration_data,  # same dict used during fitting
)
sim.etch_model.anchor_velocity_uncertainties_um_h = (
    result["anchor_velocity_uncertainties_um_h"]
)
sim.etch_model.V_max_uncertainty_um_h = result.get("V_max_uncertainty_um_h")
```

Once the uncertainties are attached, use the `_with_uncertainty` variants:

```
radius_um, radius_unc_um = sim.get_track_radius_um_with_uncertainty(
    etch_time_h=3.0,  # etching duration
    n_sigma=1.0,      # propagate +/-1 sigma of V(D) anchors
)

result = sim.get_track_length_um_with_uncertainty(
    etch_time_h=3.0,
    n_sigma=1.0,
)
if result is not None:
    depth_um, depth_unc_um = result

print(f"radius = {radius_um:.2f} +/- {radius_unc_um:.2f} um")
print(f"depth  = {depth_um:.2f} +/- {depth_unc_um:.2f} um")
```

Both methods return nan uncertainties when a bound contour cannot be extracted. `get_track_radius_um_with_uncertainty()` also floors the uncertainty at half the minimum radial grid step, since the contour extraction cannot resolve sub-cell differences.

For a full uncertainty *band* around the track contour see `get_iso_time_contour_with_uncertainty()`.

update_etch_model_and_recalculate(*new_etch_model*: `EtchRateModel` | `None` = `None`) → `None`

Replace the etch model and recompute etch-rate & arrival-time maps.

The dose map is *not* recomputed (it depends only on physics, not the etch model).

Parameters

new_etch_model (`tracketch.etching.etch_rate_model.EtchRateModel` or `None`) – New model. If `None`, recomputes using the current model (useful after in-place parameter changes).

recalculate_from_current_etch_model() → `None`

Recompute maps using the current etch model.

Alias for `update_etch_model_and_recalculate(None)`.

get_iso_time_contour(*etching_time_h*: *float*) → tuple[ndarray, ndarray]

Extract the iso-time contour from the arrival-time map.

Parameters

etching_time_h (*float*) – Etching duration in hours.

Returns

r_coords, z_coords – Coordinates along the longest contour segment.

Return type

tuple[ndarray, ndarray]

get_iso_time_contour_with_uncertainty(*etching_time_h*: *float*, *n_sigma*: *float* = 1.0, *n_band_points*: *int* = 200) → tuple[tuple[ndarray, ndarray], ndarray, ndarray, ndarray] | None

Iso-time contour with uncertainty band from V(D) anchor uncertainties.

Computes arrival-time maps for V(D) $\pm$ *n_sigma**dV, extracts the three contours, and interpolates the upper/lower bounds onto a common z grid suitable for `ax.fill_betweenx`.

Parameters

- **etching_time_h** (*float*) – Etching duration in hours.
- **n_sigma** (*float*) – Number of standard deviations to propagate (default 1).
- **n_band_points** (*int*) – Number of points on the common z grid for the band (default 200).

Returns

- tuple of ((*r_central*, *z_central*), *z_band*, *r_lo*, *r_hi*) or *None*.
- Returns **None* (and logs a warning) when the etch model carries no*
- *anchor velocity uncertainties*. Assign the output of
- `estimate_parameter_uncertainties` to
- `etch_model.anchor_velocity_uncertainties_um_h` first.

get_track_radius_um(*etch_time_h*: *float*, *threshold_percent*: *float* = 5.0) → float

Track opening radius from iso-time contour deviation.

Parameters

- **etch_time_h** (*float*) – Etching duration in hours.
- **threshold_percent** (*float*) – Deviation threshold as percentage of bulk depth (default 5 %).

Returns

Track radius in um, or nan if no contour is found.

Return type

float

get_track_radius_um_with_uncertainty(*etch_time_h*: *float*, *threshold_percent*: *float* = 5.0, *n_sigma*: *float* = 1.0) → tuple[float, float]

Track radius with uncertainty propagated from V(D) anchor uncertainties.

Evaluates the etch-rate and arrival-time maps for V(D) $\pm$ *n_sigma**dV and V(D) $\pm$ *n_sigma**dV, extracts a radius from each, and returns the central radius together with half the spread as the uncertainty estimate.

Parameters

- **etch_time_h** (*float*) – Etching duration in hours.
- **threshold_percent** (*float*) – Deviation threshold as percentage of bulk depth (default 5 %).

- **n_sigma** (*float*) – Number of standard deviations to propagate (default 1).

Returns

- **radius_um** (*float*) – Central track radius in um.
- **radius_uncertainty_um** (*float*) – Half-spread of the upper/lower bound radii (1-sigma estimate).

Raises

ValueError – If the etch model carries no anchor velocity uncertainties.

get_track_length_um_with_uncertainty(*etch_time_h: float, relative_to_surface: bool = True, n_sigma: float = 1.0*) → *tuple[float, float] | None*

Track depth with uncertainty propagated from V(D) anchor uncertainties.

Evaluates the arrival-time map for $V(D)+n_sigma*dV$ and $V(D)-n_sigma*dV$, reads the depth at $r = 0$ for each, and returns the central depth together with half the spread as the uncertainty estimate.

Parameters

- **etch_time_h** (*float*) – Etching duration in hours.
- **relative_to_surface** (*bool*) – If *True* (default), subtract the bulk-etched surface depth.
- **n_sigma** (*float*) – Number of standard deviations to propagate (default 1).

Returns

- tuple of (*depth_um, depth_uncertainty_um*) or *None*.
- *Returns *None* (and logs a warning) when the etch model carries no*
- *anchor velocity uncertainties*.

get_track_length_um(*etch_time_h: float | list, relative_to_surface: bool = True*) → *float | ndarray*

Track depth at $r = 0$ for a given etching time.

Parameters

- **etch_time_h** (*float or list[float]*) – Etching duration(s) in hours.
- **relative_to_surface** (*bool*) – If *True* (default), subtract the bulk-etched surface position so the result is the pit depth below the current surface.

Returns

Depth in um, or nan if the track does not reach $r = 0$.

Return type

float or *ndarray*

get_track_detectability(*etch_time_h: float, min_radius_um: float = 0.5, min_depth_um: float = 0.3, min_cone_half_angle_deg: float = 2.0, threshold_percent: float = 5.0*) → *dict*

Geometric detectability metrics for a given etching time.

A track is considered detectable when **all three** criteria are met:

1. $radius_um \geq min_radius_um$
2. $depth_um \geq min_depth_um$ (pit depth below bulk surface)
3. $cone_half_angle_deg \geq min_cone_half_angle_deg$

Returns

Keys: *detected, radius_um, depth_um, cone_half_angle_deg, radius_ok, depth_ok, angle_ok*.

Return type

dict

is_track_detected(*etch_time_h*: float, *min_radius_um*: float = 0.5, *min_depth_um*: float = 0.3, *min_cone_half_angle_deg*: float = 2.0, *threshold_percent*: float = 5.0) → bool

Return *True* if the track is detectable at this etching time.

Convenience wrapper around `get_track_detectability()`.

plot_iso_time_contour(*ax*: Axes, *etching_time_h*: float) → tuple[ndarray, ndarray]

Plot the iso-time contour on *ax*.

Returns

r_coords, z_coords

Return type

tuple[ndarray, ndarray]

plot_track_radius(*etch_time_h*: float, *ax*: Axes | None = None, *show_contour*: bool = True, *threshold_percent*: float = 5.0) → tuple

Plot iso-time contour with track-radius indicators.

Return type

fig, ax

plot_map(*name*: str = 'dose', *grayscale_mode*: bool = False, *plot_contours*: bool = True, *annotate_figure*: bool = True) → tuple

Plot the dose, etch-rate, or arrival-time map.

plot_LET_energy_profiles() → tuple

Plot energy and LET profiles vs. depth.

calculate_RDD_Gy(*E_MeV_u*: float) → ndarray

Calculate the radial dose distribution at a given energy.

Parameters

E_MeV_u (float) – Kinetic energy in MeV per nucleon.

Returns

Dose in Gy at each radial grid point.

Return type

ndarray

Plotting utilities for `TrackSimulator`.

`tracketch.simulation.plots.plot_map`(*etcher*: tracketch.simulation.simulator.TrackSimulator, *name*: str = 'dose', *grayscale_mode*: bool = False, *plot_contours*: bool = True, *annotate_figure*: bool = True) → tuple

Plot the dose, etch-rate, or arrival-time map.

Parameters

- **etcher** (tracketch.simulation.simulator.TrackSimulator) – Simulator instance.
- **name** (str) – 'dose', 'etch', or 'arrival'.
- **grayscale_mode** (bool) – Use greyscale colour map.
- **plot_contours** (bool) – Overlay contour lines.
- **annotate_figure** (bool) – Add CSDA-range annotation.

Returns

- **fig** (Figure)

- **ax** (*Axes*)

`tracketch.simulation.plots.plot_LET_energy_profiles`(*etcher*: `tracketch.simulation.simulator.TrackSimulator`)
→ `tuple`

Plot kinetic-energy and LET depth profiles.

Parameters

etcher (`tracketch.simulation.simulator.TrackSimulator`) – Simulator instance.

Returns

- **fig** (*Figure*)
- **axes** (*ndarray of Axes*)

Simulation grid construction and coordinate-transform helpers.

`tracketch.simulation.utils.convert_map_logspace_to_linspace`(*r_log*: `ndarray`, *z_linear*: `ndarray`, *data_map*: `ndarray`, *n_uniform_multiplier*: `int = 3`) → `tuple[ndarray, ndarray, ndarray]`

Interpolate a 2-D map from log-spaced *r* to uniform linear *r*.

Parameters

- **r_log** (`ndarray`) – Log-spaced radial coordinates (1-D).
- **z_linear** (`ndarray`) – Linear depth coordinates (1-D).
- **data_map** (`ndarray`, *shape* (`n_z`, `n_r`)) – Data on the (`z_linear`, `r_log`) grid.
- **n_uniform_multiplier** (`int`) – Resolution multiplier for the uniform grid.

Returns

- **r_uniform** (`ndarray`)
- **z_linear** (`ndarray`)
- **data_uniform** (`ndarray`)

`tracketch.simulation.utils.convert_map_linspace_to_logspace`(*r_uniform*: `ndarray`, *z_linear*: `ndarray`, *data_uniform*: `ndarray`, *r_log*: `ndarray`) → `ndarray`

Interpolate a 2-D map from uniform linear *r* back to log-spaced *r*.

Parameters

- **r_uniform** (`ndarray`) – Uniform radial coordinates.
- **z_linear** (`ndarray`) – Depth coordinates.
- **data_uniform** (`ndarray`, *shape* (`n_z`, `n_r_uniform`)) – Data on the uniform grid.
- **r_log** (`ndarray`) – Target log-spaced radial coordinates.

Return type

`ndarray`, *shape* (`n_z`, `n_r_log`)

`tracketch.simulation.utils.create_simulation_grid`(*rz_lims_dict*: `dict`) → `tuple[ndarray, ndarray, tuple[float, float], tuple[float, float]]`

Create a 2-D (*r*, *z*) simulation grid.

The radial axis is log-spaced (necessary for resolving narrow RDDs).

Parameters

rz_lims_dict (`dict`) – Grid parameters. Recognised keys: `r_min_um`, `r_max_um`, `z_min_um`, `z_max_um`, `n_points_r`, `n_points_z`.

Returns

- **r_grid_um** (*ndarray*)
- **z_grid_um** (*ndarray*)
- **r_bounds** (*tuple[float, float]*)
- **z_bounds** (*tuple[float, float]*)

Raises

TypeError – If a recognised key maps to a non-numeric value.

```
tracketch.simulation.utils.get_track_radius_from_contour(r_contour: ndarray, z_contour:
ndarray, V_bulk_um_h: float,
etch_time_h: float, threshold_percent:
float = 5.0) → float
```

Find track radius where the iso-time contour deviates from bulk etching.

Parameters

- **r_contour** (*ndarray*) – Radial coordinates of iso-time contour (um).
- **z_contour** (*ndarray*) – Depth coordinates of iso-time contour (um).
- **V_bulk_um_h** (*float*) – Bulk etch rate in um/hr.
- **etch_time_h** (*float*) – Etching time in hours.
- **threshold_percent** (*float*) – Deviation threshold (percentage of bulk depth).

Returns

Track radius in um, or nan if not found.

Return type

float

4.3 Etching

Etch rate modeling for track etching simulations.

Implements monotonic etch rate functions $V(\text{dose})$ over wide dose ranges.

```
class tracketch.etching.etch_rate_model.EtchRateModel(anchor_doses_Gy: ndarray | list[float] =
array([1.00000000e+01,
6.30957344e+02, 3.98107171e+04,
2.51188643e+06, 1.58489319e+08,
1.00000000e+10]), V_bulk_um_h: float
= 1.73, V_max_um_h: float | None =
None, V_max_uncertainty_um_h: float |
None = None, anchor_velocities_um_h:
ndarray | list[float] | None = None,
anchor_velocity_uncertainties_um_h:
ndarray | list[float] | None = None, name:
str | None = None, extrapolation_mode:
str = 'pchip', debris_alpha: float | None =
None, debris_beta: float = 1.0)
```

Bases: *object*

LOW_DOSE_ANCHOR_GY: *float* = **0.001**

summary() → *str*

Return a compact human-readable summary of model settings and ranges.

update_debris_params(*alpha*: *float* | *None*, *beta*: *float* | *None* = *None*) → *None*

Update debris-damping parameters.

Parameters

- **alpha** (*float* or *None*) – Characteristic aspect-ratio scale. *None* disables damping.
- **beta** (*float* or *None*) – Transition steepness. *None* keeps current value.

property debris_damping_enabled: *bool*

Whether debris damping is active.

is_monotonic(*num_points*: *int* = 256) → *bool*

Check monotonic non-decreasing behavior of *v*(*d*) on a dense log-dose grid.

eval(*dose_Gy*: *float* | *ndarray*) → *float* | *ndarray*

Evaluate etch rate at given dose(s).

Parameters

dose_Gy (*float* or *array-like*) – Dose in Gy.

Returns

V_um_h – Etch rate in um/hour Returns scalar if input is scalar, array if input is array
Returns **V_bulk_um_h** for NaN inputs Always >= **V_bulk_um_h** (physically consistent)

Return type

float or *ndarray*

eval_uncertainty_band(*dose_Gy*: *ndarray*, *n_sigma*: *float* = 1.0, *max_sigma_log10*: *float* = 0.5) →
tuple[*ndarray*, *ndarray*] | *None*

Return lower and upper 1-sigma etch-rate bands at the given doses.

Converts per-anchor velocity uncertainties to log10-velocity space, interpolates them onto *dose_Gy* with PCHIP (in log-dose space), and applies the band symmetrically around **eval**(*dose_Gy*) in log10 space. Results are clipped to [**V_bulk**, **V_max**].

Parameters

- **dose_Gy** (*array-like*) – Dose values at which to evaluate the band.
- **n_sigma** (*float*) – Number of standard deviations for the band width (default 1).
- **max_sigma_log10** (*float*) – Cap on interpolated sigma in log10-velocity units (default 0.5, i.e. ~factor-of-3 per sigma) to suppress artefacts from poorly-constrained high-dose anchors.

Returns

- *tuple* of (*lower*, *upper*) *ndarrays*, or *None* if no anchor uncertainties
- *are set on the model*.

update_velocities(*new_velocities_um_h*: *ndarray* | *list*[*float*]) → *None*

Update anchor velocities and recreate spline.

Accepts absolute velocities, validates them for monotonicity, and converts to relative increments internally.

Useful during optimization loops.

Parameters

new_velocities_um_h (*array-like*) – New absolute velocity values for anchor points [um/hour]

Raises

ValueError – If new velocities violate monotonicity

get_increments() → `ndarray`

Get relative velocity increments (for optimization).

Returns

increments – Relative increments [um/hour], all ≥ 0

Return type

`np.ndarray`

get_log_velocity_increments() → `ndarray`

Get monotonic increments in $\log_{10}(\text{velocity})$ space.

Defines:

$u_i = \log_{10}(v_i)$ $\Delta u_0 = u_0 - \log_{10}(V_{\text{bulk_um_h}})$ $\Delta u_i = u_i - u_{i-1}$ for $i \geq 1$

Since anchor velocities are monotonic and $\geq V_{\text{bulk_um_h}}$, all returned increments are guaranteed to be ≥ 0 .

Returns

Non-negative increments in $\log_{10}(\text{velocity})$ space.

Return type

`np.ndarray`

get_log_doses() → `ndarray`

Get $\log_{10}$ of dose anchors (for optimization in log-space).

Returns

log_doses – $\log_{10}$ of anchor doses [dimensionless]

Return type

`np.ndarray`

update_dose_anchors(anchor_doses_Gy: `ndarray` | `list[float]`) → `None`

Update dose anchor positions and recreate spline.

Validates that doses are strictly increasing.

Parameters

anchor_doses_Gy (*array-like*) – New dose anchor points [Gy]. Must be strictly increasing.

Raises

ValueError – If doses not strictly increasing or length doesn't match velocities

update_from_increments(increments_um_h: `ndarray` | `list[float]`) → `None`

Update velocities from relative increments (for optimization).

This is the recommended approach for optimization: optimize the increments directly to guarantee monotonicity by construction.

Parameters

increments_um_h (*array-like*) – Relative increments [um/hour]. Must be all ≥ 0 and have same length as anchor points.

Raises

ValueError – If increments are negative or length doesn't match If resulting velocities would be $< V_{\text{bulk_um_h}}$

update_from_log_velocity_increments(log_velocity_increments: `ndarray` | `list[float]`) → `None`

Update anchor velocities from non-negative $\log_{10}(\text{velocity})$ increments.

This parameterization is robust for wide dynamic ranges because optimization steps act multiplicatively on velocity.

Parameters

log_velocity_increments (*array-like*) – Increments in $\log_{10}(\text{velocity})$ space. Must all be ≥ 0 and have the same length as anchor points.

static log_increments_to_linear(*log_increments: ndarray | list[float], epsilon: float = 1e-06*) → ndarray

Convert log-space increments to linear (for unconstrained optimization).

For optimization, parameterize as: $\log_increments = \log(increments + \epsilon)$ This ensures increments are always positive without constraints.

Parameters

- **log_increments** (*array-like*) – Log-transformed increments (can be any real number)
- **epsilon** (*float, optional*) – Small offset to ensure positivity (default: 1e-6)

Returns

increments – Linear increments, all $> \epsilon$

Return type

np.ndarray

static linear_increments_to_log(*increments: ndarray | list[float], epsilon: float = 1e-06*) → ndarray

Convert linear increments to log-space (for unconstrained optimization).

Inverse of `log_increments_to_linear`.

Parameters

- **increments** (*array-like*) – Linear increments (must all be $> -\epsilon$)
- **epsilon** (*float, optional*) – Small offset used during forward conversion (default: 1e-6)

Returns

log_increments – Log-transformed increments

Return type

np.ndarray

static log_velocity_increments_to_linear(*optimization_parameters: ndarray | list[float], epsilon: float = 1e-12*) → ndarray

Map unconstrained optimizer parameters to positive log-velocity increments.

static linear_log_velocity_increments_to_optimization(*log_velocity_increments: ndarray | list[float], epsilon: float = 1e-12*) → ndarray

Map positive log-velocity increments to unconstrained optimizer parameters.

plot(*dose_range: tuple[float, float] = (1e-06, 1000000000000.0), n_points: int = 500, show_anchors: bool = True, show_v_bulk: bool = True, show_uncertainty_band: bool = True*) → tuple[Figure, Axes]

Plot the etch rate function.

Parameters

- **dose_range** (*tuple, optional*) – (min_dose, max_dose) in Gy for x-axis
- **n_points** (*int, optional*) – Number of evaluation points
- **show_anchors** (*bool, optional*) – Whether to show anchor points
- **show_v_bulk** (*bool, optional*) – Whether to show $V_{\text{bulk_um_h}}$ reference line

- **show_uncertainty_band** (*bool*, *optional*) – Whether to show uncertainty band if anchor uncertainties are available

Returns

fig, ax

Return type

matplotlib figure and axes

save_to_json(*filepath: str*) → *None*

Save the etch rate model to a JSON file.

Parameters**filepath** (*str*) – Path to the output JSON file**Examples**

```
>>> model.save_to_json("my_model.json")
```

classmethod load_from_json(*filepath: str*, *verbose: bool = False*) → *EtchRateModel*

Load an etch rate model from a JSON file.

Parameters**filepath** (*str*) – Path to the JSON file**Returns**

The loaded model

Return type*EtchRateModel***Examples**

```
>>> model = EtchRateModel.load_from_json("my_model.json")
```

Etch-rate model persistence and default-model factory.

Provides `save_etchrate_model()`, `load_etchrate_model()`, and a convenience factory `default_etch_rate_model()`.

```
tracketch.etching.etch_rate_model_io.default_etch_rate_model(n_anchors: int = 15,
                                                             bulk_rate_um_h: float = 1.73)
                                                             → EtchRateModel
```

Generate a generic etch-rate model initialised from Doerschel data.

Parameters

- **n_anchors** (*int*) – Number of fine-resolution anchor points in the high-dose region.
- **bulk_rate_um_h** (*float*) – Bulk etch rate in um/hr.

Return type*tracketch.etching.etch_rate_model.EtchRateModel***tracketch.etching.etch_rate_model_io.get_model_path**(*model_name: str*) → *Path*

Get the full path to a model file based on config.

Parameters**model_name** (*str*) – The model identifier (e.g., 'Doerschel_etching')**Returns**

Full path to the model file

Return type*Path*

Examples

```
>>> path = get_model_path('Doerschel_etching')
```

```
tracketch.etching.etch_rate_model_io.save_etchrate_model(model: EtchRateModel, model_name: str) → None
```

Save a model using the configured path.

Parameters

- **model** (`tracketch.etching.etch_rate_model.EtchRateModel`) – The model to save
- **model_name** (`str`) – The model identifier (e.g., 'Doerschel_etching')

Examples

```
>>> save_etchrate_model(my_model, 'my_custom_model')
```

```
tracketch.etching.etch_rate_model_io.load_etchrate_model(model_name: str) → EtchRateModel
```

Load a model using the configured path.

Parameters

model_name (`str`) – The model identifier (e.g., 'Doerschel_etching')

Returns

The loaded model

Return type

`tracketch.etching.etch_rate_model.EtchRateModel`

Examples

```
>>> model = load_etchrate_model('Doerschel_etching')
```

4.4 Physics

Ion physics: RDD, LET, CSDA range, and energy-loss utilities.

```
tracketch.physics.get_RDD_Gy(r_m: ndarray, E_MeV_u: float, particle_name: str, RDD_name: str = 'Cucinotta', material_name: str = 'CR39', core_nm: float | None = None, normalise_to_LET: bool | None = None, LET_source: str = 'SRIM', verbose: bool = False) → ndarray
```

Radial dose distribution scaled to *material_name*.

The RDD is computed for water (via libamtrack) and then corrected:

1. Radii are density-scaled to account for different electron ranges.
2. A dose correction restores the radial integral after the range shift.
3. A stopping-power ratio (SPR) converts dose-to-water into dose-to- target-material.
4. Optionally, the profile is renormalised so that its radial integral equals the target-material LET.

Parameters

- **r_m** (`ndarray`) – Radial distances in metres.
- **E_MeV_u** (`float`) – Kinetic energy in MeV/u.
- **particle_name** (`str`) – Ion species.
- **RDD_name** (`str`) – RDD model name (default "Cucinotta").

- **material_name** (*str*) – Target material (default "CR39").
- **core_nm** (*float* or *None*) – Core radius override in nm.
- **normalise_to_LET** (*bool* or *None*) – Force LET normalisation. *None* uses the model default.
- **LET_source** (*str*) – Stopping-power source for the SPR (default "SRIM").
- **verbose** (*bool*) – Print diagnostic information.

Returns

Dose in Gy at each radius.

Return type

ndarray

```
tracketch.physics.get_LET_keV_um(energy_MeV_u: float | ndarray, particle_name: str, material_name: str, source: str = 'SRIM') → float | ndarray
```

Return LET in keV/um from the chosen stopping-power database.

Parameters

- **energy_MeV_u** (*float* or *ndarray*) – Kinetic energy in MeV per nucleon.
- **particle_name** (*str*) – Ion species, e.g. "12C".
- **material_name** (*str*) – Target material, e.g. "CR39".
- **source** (*str*) – "SRIM" (default) or "libamtrack".

Returns

LET in keV/um.

Return type

float or ndarray

Raises

ValueError – If *source* or *material_name* are not recognised.

```
tracketch.physics.get_CSDA_um(energy_MeV_u: float, particle_name: str, material_name: str, source: str = 'SRIM') → float
```

Return the CSDA range in um from the chosen stopping-power database.

Parameters

- **energy_MeV_u** (*float*) – Kinetic energy in MeV/u.
- **particle_name** (*str*) – Ion species.
- **material_name** (*str*) – Target material.
- **source** (*str*) – "SRIM" (default) or "libamtrack".

Returns

CSDA range in um.

Return type

float

Raises

ValueError – If *source* or *material_name* are not recognised.

```
tracketch.physics.get_dose_map_Gy(z_um: ndarray, r_um: ndarray, energy_MeV_u_array: ndarray, particle_name: str, material_name: str = 'CR39', RDD_name: str = 'Cucinotta', LET_source: str = 'SRIM', core_radius_nm: float | None = None, normalise_to_LET: bool | None = None, verbose: bool = False, n_jobs: int = 1) → ndarray
```

Build a 2-D dose map by evaluating the RDD at each depth step.

Parameters

- **z_um** (*ndarray*, *shape* (*n_z*,)) – Depth coordinates in um.
- **r_um** (*ndarray*, *shape* (*n_r*,)) – Radial coordinates in um.
- **energy_MeV_u_array** (*ndarray*, *shape* (*n_z*,)) – Ion energy at each depth (from [`get\_LET\_energy\_profile\(\)`](#)).
- **particle_name** (*str*) – Ion species.
- **material_name** (*str*) – Target material.
- **RDD_name** (*str*) – RDD model name.
- **LET_source** (*str*) – Stopping-power source.
- **core_radius_nm** (*float* or *None*) – Core radius override.
- **normalise_to_LET** (*bool* or *None*) – Force LET normalisation.
- **verbose** (*bool*) – Print diagnostics.
- **n_jobs** (*int*) – Number of parallel workers for RDD evaluation. -1 uses all available CPU cores (via `joblib`). Default 1 runs sequentially.

Returns

Local dose in Gy.

Return type

ndarray, *shape* (*n_z*, *n_r*)

`tracketch.physics.get_mean_core_value`(*metric_map*: *ndarray*, *r_um*: *ndarray*, *core_r_threshold_um*: *float* = 0.1, *min_core_r_um*: *float* = 0.001) → *ndarray*

Average a map over the track core (small-*r* region) at each depth.

Parameters

- **metric_map** (*ndarray*, *shape* (*n_z*, *n_r*)) – Dose or etch-rate map.
- **r_um** (*ndarray*, *shape* (*n_r*,)) – Radial coordinates in um.
- **core_r_threshold_um** (*float*) – Outer boundary of the core region in um.
- **min_core_r_um** (*float*) – Inner boundary of the core region in um.

Returns

Mean value within the core at each depth. nan where no valid data exists.

Return type

ndarray, *shape* (*n_z*,)

`tracketch.physics.RDD_integral`(*r_m*: *ndarray*, *D_Gy*: *ndarray*, *density_g_cm3*: *float* | *None* = *None*) → *float*

Radial integral of the dose distribution.

Computes

$$\int 2\pi r D(r) dr$$

and converts to keV/um (per unit density) via standard unit factors.

Parameters

- **r_m** (*ndarray*) – Radial positions in metres.
- **D_Gy** (*ndarray*) – Dose in Gy at each radius.
- **density_g_cm3** (*float* or *None*) – If given, multiply by the density in kg/m³.

Returns

Integral value (keV/um if density is provided in kg/m³ upstream).

Return type

float

`tracketch.physics.convert_MeV_to_MeV_u(Energy_MeV: float, particle_name: str) → float`

Convert total kinetic energy (MeV) to energy per nucleon (MeV/u).

Parameters

- **Energy_MeV** (float) – Total kinetic energy in MeV.
- **particle_name** (str) – Ion species identifier.

Returns

Energy in MeV/u.

Return type

float

`tracketch.physics.convert_MeV_u_to_MeV(Energy_MeV_u: float, particle_name: str) → float`

Convert energy per nucleon (MeV/u) to total kinetic energy (MeV).

Parameters

- **Energy_MeV_u** (float) – Energy in MeV per nucleon.
- **particle_name** (str) – Ion species identifier.

Returns

Total kinetic energy in MeV.

Return type

float

`tracketch.physics.get_energy_after_slab_MeV_u(E_initial_MeV_u: float, particle_name: str, slab_thickness_um: float, material_name: str = 'CR39', source: str = 'SRIM') → float`

Residual energy after traversing a slab of given thickness.

Steps through the slab in small increments, computing the LET at each step to update the energy.

Parameters

- **E_initial_MeV_u** (float) – Incident energy in MeV/u.
- **particle_name** (str) – Ion species.
- **slab_thickness_um** (float) – Slab thickness in um.
- **material_name** (str) – Target material (default "CR39").
- **source** (str) – Stopping-power source (default "SRIM").

Returns

Residual energy in MeV/u (0 if the ion stops).

Return type

float

`tracketch.physics.apply_range_straggling(dz_um_list: ndarray, metric: ndarray, longitudinal_straggling_um: float, n_sigma: float = 1) → ndarray`

Broaden a depth profile by Gaussian range-straggling convolution.

Parameters

- **dz_um_list** (ndarray) – Depth grid in um (assumed uniform spacing).

- **metric** (*ndarray*) – Profile to broaden (same length as *dz_um_list*).
- **longitudinal_straggling_um** (*float*) – 1-sigma straggling width in um.
- **n_sigma** (*float*) – Number of sigma to apply (0 = no straggling).

Returns

Broadened profile.

Return type

ndarray

`tracketch.physics.get_range_straggling_um(energy_MeV_u: float, particle_name: str, material_name: str) → float`

Longitudinal range straggling from SRIM data.

Parameters

- **energy_MeV_u** (*float*) – Kinetic energy in MeV/u.
- **particle_name** (*str*) – Ion species.
- **material_name** (*str*) – Target material.

Returns

Straggling in um. Returns 0 if the particle is not available in SRIM (justified for heavy ions with negligible straggling).

Return type

float

`tracketch.physics.get_LET_energy_profile(dz_um_list: ndarray, energy_MeV_u: float, particle_name: str, material_name: str, range_straggling: bool = True, n_straggling_sigma: float = 1, source: str = 'SRIM') → tuple[ndarray, ndarray]`

Compute energy and LET profiles along depth.

For source="SRIM" the profiles are computed analytically by inverting the CSDA-range table (vectorised, no stepping loop). For source="libamtrack" the original iterative stepping is used.

Optionally applies longitudinal range straggling as a Gaussian convolution.

Parameters

- **dz_um_list** (*ndarray*) – Depth grid in um.
- **energy_MeV_u** (*float*) – Incident energy in MeV/u.
- **particle_name** (*str*) – Ion species.
- **material_name** (*str*) – Target material.
- **range_straggling** (*bool*) – Apply Gaussian straggling (default True).
- **n_straggling_sigma** (*float*) – Number of straggling sigma (default 1).
- **source** (*str*) – Stopping-power database (default "SRIM").

Returns

- **energy_profile** (*ndarray*) – Kinetic energy in MeV/u at each depth.
- **LET_profile** (*ndarray*) – LET in keV/um at each depth.

Interface to libamtrack for RDD, LET, and CSDA calculations.

CR-39 is not defined in libamtrack. All calculations are performed for water (or PMMA for density-matching) and then scaled to CR-39 by the caller.

`tracketch.physics.libamtrack.get_material_number_and_density(density_g_cm3: float) → tuple[int, float]`

Return the libamtrack material number closest to the requested density.

Parameters

density_g_cm3 (*float*) – Target density in g/cm³. Must be in (0, 10].

Returns

- **material_no** (*int*) – libamtrack material number.
- **material_density_g_cm3** (*float*) – Actual density of the matched material.

Raises

ValueError – If *density_g_cm3* is outside (0, 10].

`tracketch.physics.libamtrack.get_LET_keV_um_libam(particle_name: str, E_MeV_u: float, density_g_cm3: float = 1.31) → float`

Compute LET in keV/um for a given ion and energy via libamtrack.

The stopping power is computed for the closest libamtrack material and then linearly scaled to *density_g_cm3*.

Parameters

- **particle_name** (*str*) – Ion species, e.g. "12C".
- **E_MeV_u** (*float*) – Kinetic energy in MeV per nucleon.
- **density_g_cm3** (*float*) – Target material density (default 1.31, CR-39).

Returns

LET in keV/um.

Return type

float

`tracketch.physics.libamtrack.RDD_libamtrack_Gy(r_m: ndarray, E_MeV_u: float, particle_name: str, RDD_name: str = 'Cucinotta', core_nm: float | None = None) → ndarray`

Compute radial dose distribution in Gy at radii *r_m* (metres).

Uses libamtrack for **water**. Material scaling must be done by the caller (see `tracketch.physics.get_RDD_Gy()`).

Parameters

- **r_m** (*ndarray*) – Radial distances in metres.
- **E_MeV_u** (*float*) – Kinetic energy in MeV/u.
- **particle_name** (*str*) – Ion species.
- **RDD_name** (*str*) – RDD model name (default "Cucinotta").
- **core_nm** (*float* or *None*) – Core radius override in nm. None uses the model default.

Returns

Dose in Gy. Values below *d_min_Gy* are set to nan.

Return type

ndarray

Raises

ValueError – If *RDD_name* is not defined in the configuration.

`tracketch.physics.libamtrack.get_energy_after_slab_MeV_um_libam(E_initial_MeV_u: float, particle_name: str, slab_thickness_um: float, material_name: str) → float`

Residual energy after traversing a slab, computed via libamtrack.

Only works for materials defined in libamtrack ("Liquid, Water" or "PMMA").

Parameters

- **E_initial_MeV_u** (*float*) – Incident energy in MeV/u.
- **particle_name** (*str*) – Ion species.
- **slab_thickness_um** (*float*) – Slab thickness in um.
- **material_name** (*str*) – "Liquid, Water" or "PMMA".

Returns

Energy after the slab in MeV/u.

Return type

float

Raises

ValueError – If *material_name* is not supported.

`tracketch.physics.libamtrack.get_CSDA_range_um_libam(particle_name: str, E_MeV_u: float,
 density_g_cm3: float = 1.31) → float`

CSDA range in um via libamtrack, scaled to the target density.

libamtrack returns a density-normalised range (g/cm²), which is converted to um using *density_g_cm3*.

Parameters

- **particle_name** (*str*) – Ion species.
- **E_MeV_u** (*float*) – Kinetic energy in MeV/u.
- **density_g_cm3** (*float*) – Target material density (default 1.31, CR-39).

Returns

CSDA range in um.

Return type

float

High-level ion-physics utilities.

Wrappers around SRIM tables and libamtrack that provide:

- LET and CSDA range lookups (multi-source),
- radial dose distributions (RDD) scaled to arbitrary materials,
- dose-map construction,
- energy-loss stepping, and
- longitudinal range-straggling convolution.

`tracketch.physics.utils_ions.get_material_config()`

Return the materials section of the material configuration (cached).

`tracketch.physics.utils_ions.get_LET_keV_um(energy_MeV_u: float | ndarray, particle_name: str,
 material_name: str, source: str = 'SRIM') → float |
 ndarray`

Return LET in keV/um from the chosen stopping-power database.

Parameters

- **energy_MeV_u** (*float* or *ndarray*) – Kinetic energy in MeV per nucleon.
- **particle_name** (*str*) – Ion species, e.g. "12C".

- **material_name** (*str*) – Target material, e.g. "CR39".
- **source** (*str*) – "SRIM" (default) or "libamtrack".

Returns

LET in keV/um.

Return type

float or *ndarray*

Raises

ValueError – If *source* or *material_name* are not recognised.

```
tracketch.physics.utils_ions.get_RDD_Gy(r_m: ndarray, E_MeV_u: float, particle_name: str,
                                         RDD_name: str = 'Cucinotta', material_name: str =
                                         'CR39', core_nm: float | None = None, normalise_to_LET:
                                         bool | None = None, LET_source: str = 'SRIM', verbose:
                                         bool = False) → ndarray
```

Radial dose distribution scaled to *material_name*.

The RDD is computed for water (via libamtrack) and then corrected:

1. Radii are density-scaled to account for different electron ranges.
2. A dose correction restores the radial integral after the range shift.
3. A stopping-power ratio (SPR) converts dose-to-water into dose-to- target-material.
4. Optionally, the profile is renormalised so that its radial integral equals the target-material LET.

Parameters

- **r_m** (*ndarray*) – Radial distances in metres.
- **E_MeV_u** (*float*) – Kinetic energy in MeV/u.
- **particle_name** (*str*) – Ion species.
- **RDD_name** (*str*) – RDD model name (default "Cucinotta").
- **material_name** (*str*) – Target material (default "CR39").
- **core_nm** (*float* or *None*) – Core radius override in nm.
- **normalise_to_LET** (*bool* or *None*) – Force LET normalisation. *None* uses the model default.
- **LET_source** (*str*) – Stopping-power source for the SPR (default "SRIM").
- **verbose** (*bool*) – Print diagnostic information.

Returns

Dose in Gy at each radius.

Return type

ndarray

```
tracketch.physics.utils_ions.RDD_integral(r_m: ndarray, D_Gy: ndarray, density_g_cm3: float |
                                          None = None) → float
```

Radial integral of the dose distribution.

Computes

$$\int 2\pi r D(r) dr$$

and converts to keV/um (per unit density) via standard unit factors.

Parameters

- **r_m** (*ndarray*) – Radial positions in metres.
- **D_Gy** (*ndarray*) – Dose in Gy at each radius.
- **density_g_cm3** (*float* or *None*) – If given, multiply by the density in kg/m³.

Returns

Integral value (keV/um if density is provided in kg/m³ upstream).

Return type

float

`tracketch.physics.utils_ions.get_CSDA_um(energy_MeV_u: float, particle_name: str, material_name: str, source: str = 'SRIM') → float`

Return the CSDA range in um from the chosen stopping-power database.

Parameters

- **energy_MeV_u** (*float*) – Kinetic energy in MeV/u.
- **particle_name** (*str*) – Ion species.
- **material_name** (*str*) – Target material.
- **source** (*str*) – "SRIM" (default) or "libamtrack".

Returns

CSDA range in um.

Return type

float

Raises

ValueError – If *source* or *material_name* are not recognised.

`tracketch.physics.utils_ions.convert_MeV_to_MeV_u(Energy_MeV: float, particle_name: str) → float`

Convert total kinetic energy (MeV) to energy per nucleon (MeV/u).

Parameters

- **Energy_MeV** (*float*) – Total kinetic energy in MeV.
- **particle_name** (*str*) – Ion species identifier.

Returns

Energy in MeV/u.

Return type

float

`tracketch.physics.utils_ions.convert_MeV_u_to_MeV(Energy_MeV_u: float, particle_name: str) → float`

Convert energy per nucleon (MeV/u) to total kinetic energy (MeV).

Parameters

- **Energy_MeV_u** (*float*) – Energy in MeV per nucleon.
- **particle_name** (*str*) – Ion species identifier.

Returns

Total kinetic energy in MeV.

Return type

float

```
tracketch.physics.utils_ions.get_energy_after_slab_MeV_u(E_initial_MeV_u: float,
                                                         particle_name: str,
                                                         slab_thickness_um: float,
                                                         material_name: str = 'CR39', source:
                                                         str = 'SRIM') → float
```

Residual energy after traversing a slab of given thickness.

Steps through the slab in small increments, computing the LET at each step to update the energy.

Parameters

- **E_initial_MeV_u** (*float*) – Incident energy in MeV/u.
- **particle_name** (*str*) – Ion species.
- **slab_thickness_um** (*float*) – Slab thickness in um.
- **material_name** (*str*) – Target material (default "CR39").
- **source** (*str*) – Stopping-power source (default "SRIM").

Returns

Residual energy in MeV/u (0 if the ion stops).

Return type

float

```
tracketch.physics.utils_ions.get_dose_map_Gy(z_um: ndarray, r_um: ndarray,
                                              energy_MeV_u_array: ndarray, particle_name: str,
                                              material_name: str = 'CR39', RDD_name: str =
                                              'Cucinotta', LET_source: str = 'SRIM',
                                              core_radius_nm: float | None = None,
                                              normalise_to_LET: bool | None = None, verbose:
                                              bool = False, n_jobs: int = 1) → ndarray
```

Build a 2-D dose map by evaluating the RDD at each depth step.

Parameters

- **z_um** (*ndarray*, *shape* (*n_z*,)) – Depth coordinates in um.
- **r_um** (*ndarray*, *shape* (*n_r*,)) – Radial coordinates in um.
- **energy_MeV_u_array** (*ndarray*, *shape* (*n_z*,)) – Ion energy at each depth (from *get_LET_energy_profile()*).
- **particle_name** (*str*) – Ion species.
- **material_name** (*str*) – Target material.
- **RDD_name** (*str*) – RDD model name.
- **LET_source** (*str*) – Stopping-power source.
- **core_radius_nm** (*float* or *None*) – Core radius override.
- **normalise_to_LET** (*bool* or *None*) – Force LET normalisation.
- **verbose** (*bool*) – Print diagnostics.
- **n_jobs** (*int*) – Number of parallel workers for RDD evaluation. -1 uses all available CPU cores (via *joblib*). Default 1 runs sequentially.

Returns

Local dose in Gy.

Return type

ndarray, *shape* (*n_z*, *n_r*)

```
tracketch.physics.utils_ions.get_mean_core_value(metric_map: ndarray, r_um: ndarray,  
                                                  core_r_threshold_um: float = 0.1,  
                                                  min_core_r_um: float = 0.001) → ndarray
```

Average a map over the track core (small- r region) at each depth.

Parameters

- **metric_map** (*ndarray*, *shape* (n_z , n_r)) – Dose or etch-rate map.
- **r_um** (*ndarray*, *shape* (n_r ,)) – Radial coordinates in um.
- **core_r_threshold_um** (*float*) – Outer boundary of the core region in um.
- **min_core_r_um** (*float*) – Inner boundary of the core region in um.

Returns

Mean value within the core at each depth. nan where no valid data exists.

Return type

ndarray, shape (n_z ,)

```
tracketch.physics.utils_ions.get_range_stragglng_um(energy_MeV_u: float, particle_name: str,  
                                                    material_name: str) → float
```

Longitudinal range stragglng from SRIM data.

Parameters

- **energy_MeV_u** (*float*) – Kinetic energy in MeV/u.
- **particle_name** (*str*) – Ion species.
- **material_name** (*str*) – Target material.

Returns

Stragglng in um. Returns 0 if the particle is not available in SRIM (justified for heavy ions with negligible stragglng).

Return type

float

```
tracketch.physics.utils_ions.apply_range_stragglng(dz_um_list: ndarray, metric: ndarray,  
                                                    longitudinal_stragglng_um: float, n_sigma:  
                                                    float = 1) → ndarray
```

Broaden a depth profile by Gaussian range-stragglng convolution.

Parameters

- **dz_um_list** (*ndarray*) – Depth grid in um (assumed uniform spacing).
- **metric** (*ndarray*) – Profile to broaden (same length as *dz_um_list*).
- **longitudinal_stragglng_um** (*float*) – 1-sigma stragglng width in um.
- **n_sigma** (*float*) – Number of sigma to apply (0 = no stragglng).

Returns

Broadened profile.

Return type

ndarray

```
tracketch.physics.utils_ions.get_LET_energy_profile(dz_um_list: ndarray, energy_MeV_u: float,  
                                                    particle_name: str, material_name: str,  
                                                    range_stragglng: bool = True,  
                                                    n_stragglng_sigma: float = 1, source: str =  
                                                    'SRIM') → tuple[ndarray, ndarray]
```

Compute energy and LET profiles along depth.

For source="SRIM" the profiles are computed analytically by inverting the CSDA-range table (vectorised, no stepping loop). For source="libamtrack" the original iterative stepping is used.

Optionally applies longitudinal range straggling as a Gaussian convolution.

Parameters

- **dz_um_list** (*ndarray*) – Depth grid in um.
- **energy_MeV_u** (*float*) – Incident energy in MeV/u.
- **particle_name** (*str*) – Ion species.
- **material_name** (*str*) – Target material.
- **range_straggling** (*bool*) – Apply Gaussian straggling (default True).
- **n_straggling_sigma** (*float*) – Number of straggling sigma (default 1).
- **source** (*str*) – Stopping-power database (default "SRIM").

Returns

- **energy_profile** (*ndarray*) – Kinetic energy in MeV/u at each depth.
- **LET_profile** (*ndarray*) – LET in keV/um at each depth.

```
tracketch.physics.utils_ions.get_f_dose_distribution(E_MeV_u: float, particle_name: str,
                                                    RDD_name: str = 'Cucinotta',
                                                    material_name: str = 'CR39', core_nm:
                                                    float | None = None, LET_source: str =
                                                    'SRIM', f_cutoff: float = 1e-15, d_min_Gy:
                                                    float = 1e-05, r_min_m: float = 5e-11,
                                                    r_max_m: float = 0.01, npoints: int =
                                                    1000) → tuple[ndarray, ndarray, ndarray]
```

Compute the local-dose probability density $f(d)$ for an ion track.

The PDF describes the fraction of the track cross-section receiving a local dose between d and $d + dd$. Derived from the RDD by inverting $r(d)$ and differentiating the cumulative area fraction.

Parameters

- **E_MeV_u** (*float*) – Kinetic energy in MeV/u.
- **particle_name** (*str*) – Ion species.
- **RDD_name** (*str*) – RDD model name.
- **material_name** (*str*) – Target material.
- **core_nm** (*float* or *None*) – Core radius override.
- **LET_source** (*str*) – Stopping-power source.
- **f_cutoff** (*float*) – Bins where $f \cdot d < f_cutoff$ are discarded.
- **d_min_Gy** (*float*) – Minimum dose threshold.
- **r_min_m** (*float*) – Inner radial sampling bound in metres.
- **r_max_m** (*float*) – Outer radial sampling bound in metres.
- **npoints** (*int*) – Number of radial sample points.

Returns

- **f** (*ndarray*) – Normalised dose PDF in Gy^{-1} .
- **midpoints_Gy** (*ndarray*) – Dose bin midpoints in Gy.
- **dd_Gy** (*ndarray*) – Dose bin widths in Gy.

Raises

ValueError – If too few valid RDD points are found.

4.5 Wavefront

Arrival time computation methods for track etching simulation.

This package provides multiple algorithms for computing arrival times on 2D non-uniform (e.g., log-spaced radial, linear depth) grids:

- FMM (Fast Marching Method): For uniform grids
- Dijkstra: For non-uniform grids, with Numba or C++ backends

4.5.1 Main algorithms

FMM (`wavefront.fmm`)

Fast Marching Method for smooth speed maps. Requires uniform grid (or interpolation from log-spaced grid).

Dijkstra (`wavefront.dijkstra`)

Dijkstra's algorithm optimized for non-uniform grids using 8-connected neighborhood. Available backends:

- `dijkstra_numba` (default): Numba JIT-compiled, ~5-20x speedup
- `dijkstra_cpp` (optional): C++ with pybind11, ~50-100x speedup

Examples

Using Dijkstra with log-spaced grid (typical):

```
from tracketch.wavefront.dijkstra import arrival_time_dijkstra_fast

arrival_map = arrival_time_dijkstra_fast(
    r_log_um, z_um, etch_rate_map
)
```

Using FMM with interpolation to uniform grid:

```
from tracketch.wavefront.fmm import arrival_time_fmm

arrival_map = arrival_time_fmm(
    r_log_um, z_um, etch_rate_map, r_is_logscaled=True
)
```

Using C++ Dijkstra (if compiled):

```
from tracketch.wavefront.dijkstra import arrival_time_dijkstra_cpp

arrival_map = arrival_time_dijkstra_cpp(
    r_log_um, z_um, etch_rate_map
)
```

```
tracketch.wavefront.arrival_time_dijkstra_fast(r_um: ndarray, z_um: ndarray, etch_rate_map:
                                                ndarray, r_is_logscaled: bool = True, connectivity:
                                                Literal[8, 16, 32] = 8) → ndarray
```

Compute arrival time using Dijkstra's algorithm with Numba-accelerated graph construction.

Parameters

- **r_um** (*np.ndarray*) – Radial coordinates (can be non-uniform/log-spaced)

- **z_um** (*np.ndarray*) – Depth coordinates (linear)
- **etch_rate_map** (*np.ndarray*) – Etch rate (speed) at each (z, r) point, shape (n_z, n_r)
- **r_is_logscaled** (*bool*) – Whether r grid is log-spaced (informational only)
- **connectivity** (*{8, 16, 32}*) – Number of neighbor connections per node. - 8: Standard (0deg, 45deg, 90deg, ...) - fastest, ~2% metrication error - 16: Knight's move added (~1% error) - 32: Extended knight's moves (~0.5% error)

Returns

arrival_time_map – Arrival time at each grid point, shape (n_z, n_r)

Return type

np.ndarray

```
tracketch.wavefront.arrival_time_dijkstra(r_um: ndarray, z_um: ndarray, etch_rate_map: ndarray,
                                         r_is_logscaled: bool = True, connectivity: Literal[8, 16,
                                         32] = 8, backend: Literal['auto', 'cpp', 'numba'] = 'auto')
                                         → ndarray
```

Compute arrival times using Dijkstra's algorithm.

Auto-selects the fastest available backend (C++ > Numba).

Parameters

- **r_um** (*np.ndarray*) – Radial coordinates (can be non-uniform/log-spaced)
- **z_um** (*np.ndarray*) – Depth coordinates (linear)
- **etch_rate_map** (*np.ndarray*) – Etch rate (speed) at each (z, r) point, shape (n_z, n_r)
- **r_is_logscaled** (*bool*) – Whether r grid is log-spaced (informational only)
- **connectivity** (*{8, 16, 32}*) – Number of neighbor connections per node. - 8: Standard (fastest, ~2% metrication error) - 16: Knight's moves added (~1% error) - 32: Extended knight's moves (~0.5% error)
- **backend** (*{'auto', 'cpp', 'numba'}*) – Which backend to use. “auto” selects C++ if available, else Numba.

Returns

arrival_time_map – Arrival time at each grid point, shape (n_z, n_r)

Return type

np.ndarray

```
tracketch.wavefront.arrival_time_fmm(r_um: ndarray, z_um: ndarray, etch_rate_map: ndarray,
                                     r_is_logscaled: bool = True, theta_deg: float = 0.0,
                                     n_uniform_multiplier: int = 3)
```

Calculate arrival time map using fast marching method.

Parameters

- **r_um** (*np.ndarray*) – Radial grid (may be log-scaled)
- **z_um** (*np.ndarray*) – Depth grid (linear)
- **etch_rate_map** (*np.ndarray*) – Etch rate at each (z, r) point, shape (n_z, n_r)
- **r_is_logscaled** (*bool*, *optional*) – Whether r grid is log-scaled (default: True)
- **theta_deg** (*float*, *optional*) – Track angle relative to surface normal in degrees (default: 0.0). theta=0: track perpendicular to surface (wavefront starts at z=0) theta>0: tilted track, wavefront starts along z = r * tan(theta)
- **n_uniform_multiplier** (*int*, *optional*) – Multiplier for number of uniform grid points (default: 3). Higher values better preserve variation from log grid, especially at small radii. Set to 1 to use same number of points as original grid.

Returns

arrival_time_map – Arrival time at each (z, r) point

Return type

np.ndarray

Arrival-time map dispatcher and iso-time contour extraction.

```
tracketch.wavefront.utils_march.get_arrival_time_map(r_um: ndarray, z_um: ndarray,
                                                    etch_rate_map: ndarray, method: str =
                                                    'dijkstra', r_is_logscaled: bool = True,
                                                    theta_deg: float = 0.0,
                                                    n_uniform_multiplier: int = 3,
                                                    connectivity: Literal[8, 16, 32] = 8) →
                                                    ndarray
```

Calculate the arrival-time map using the specified method.

Parameters

- **r_um** (ndarray) – Radial coordinates in um.
- **z_um** (ndarray) – Axial coordinates in um.
- **etch_rate_map** (ndarray) – Etch rates in um/hr, shape (n_z, n_r).
- **method** (str) – "dijkstra_cpp" (fastest), "dijkstra_numba", or "fmm" (supports tilted wavefronts).
- **r_is_logscaled** (bool) – Whether the radial coordinates are log-spaced.
- **theta_deg** (float) – Track angle relative to the surface normal in degrees.
- **n_uniform_multiplier** (int) – FMM only – multiplier for the uniform-grid resolution.
- **connectivity** ({8, 16, 32}) – Dijkstra only – neighbour connectivity.

Returns

Arrival-time map in hours, same shape as *etch_rate_map*.

Return type

ndarray

```
tracketch.wavefront.utils_march.get_iso_time_contour(arrival_time_map: ndarray, r_um:
                                                    ndarray, z_um: ndarray, etching_time_h:
                                                    float) → tuple[ndarray, ndarray]
```

Extract the iso-time contour from the arrival-time map.

Parameters

- **arrival_time_map** (ndarray, shape (n_z, n_r)) – Arrival times in hours.
- **r_um** (ndarray) – Radial coordinates (may be log-spaced).
- **z_um** (ndarray) – Depth coordinates (linear).
- **etching_time_h** (float) – Target arrival time in hours.

Returns

- **r_coords** (ndarray)
- **z_coords** (ndarray) – Coordinates along the longest contour segment.

4.6 Stopping Power

SRIM output parsing, interpolation factories, and bragg-variant selection.

`tracketch.physics.SRIM.SRIM_lib.get_bragg_correction() → int`

Return the active bragg correction percentage.

`tracketch.physics.SRIM.SRIM_lib.set_bragg_correction(pct: int) → None`

Set the active bragg correction percentage and clear interpolation caches.

`tracketch.physics.SRIM.SRIM_lib.SRIM_MeV_u_to_LET(particle_name: str, material_name: str) → interp1d`

Return an interpolator: energy (MeV/u) -> LET (keV/um).

`tracketch.physics.SRIM.SRIM_lib.SRIM_LET_to_MeV_u(particle_name: str, material_name: str) → interp1d`

Return an interpolator: LET (keV/um) -> energy (MeV/u).

`tracketch.physics.SRIM.SRIM_lib.SRIM_MeV_u_to_CSDA_um(particle_name: str, material_name: str) → interp1d`

Return an interpolator: energy (MeV/u) -> CSDA range (um).

`tracketch.physics.SRIM.SRIM_lib.SRIM_CSDA_um_to_MeV_u(particle_name: str, material_name: str) → interp1d`

Return an interpolator: CSDA range (um) -> energy (MeV/u).

`tracketch.physics.SRIM.SRIM_lib.load_SRIM_cfg()`

Load SRIM configuration (cached).

`tracketch.physics.SRIM.SRIM_lib.get_SRIM_df_folder(bragg_correction: int | None = None) → Path`

Return the processed CSV folder for a bragg correction variant.

`tracketch.physics.SRIM.SRIM_lib.get_SRIM_raw_folder(bragg_correction: int | None = None) → Path`

Return the raw .dat folder for a bragg correction variant.

`tracketch.physics.SRIM.SRIM_lib.get_SRIM_df(particle_name: str, material_name: str | None = None) → DataFrame`

Load pre-processed SRIM DataFrame for the given particle.

`tracketch.physics.SRIM.SRIM_lib.convert_SRIM_distance_to_um(range_unit: str, item: str) → float`

Convert a SRIM distance value to micrometres.

`tracketch.physics.SRIM.SRIM_lib.convert_SRIM_output_to_df(particle_name: str, material_name: str, SRIM_folder: str | Path) → DataFrame`

Parse a raw SRIM .dat output file into a tidy DataFrame.

`tracketch.physics.SRIM.SRIM_lib.generate_SRIM_dfs(bragg_correction: int, SRIM_folder: str | Path | None = None) → None`

Generate pre-processed CSV DataFrames for one bragg correction variant.

INDICES AND TABLES

- `genindex`
- `modindex`
- `search`

PYTHON MODULE INDEX

t

- `tracketch`, [17](#)
- `tracketch.etching.etch_rate_model`, [37](#)
- `tracketch.etching.etch_rate_model_io`, [41](#)
- `tracketch.physics`, [42](#)
- `tracketch.physics.libamtrack`, [46](#)
- `tracketch.physics.SRIM.SRIM_lib`, [57](#)
- `tracketch.physics.utils_ions`, [48](#)
- `tracketch.simulation.plots`, [35](#)
- `tracketch.simulation.simulator`, [29](#)
- `tracketch.simulation.utils`, [36](#)
- `tracketch.wavefront`, [54](#)
- `tracketch.wavefront.utils_march`, [56](#)

A

`anchor_velocities_um_h` (*tracketch.EtchRateModel* attribute), 27

`anchor_velocity_uncertainties_um_h` (*tracketch.EtchRateModel* attribute), 27

`apply_range_stragglings()` (in module *tracketch.physics*), 45

`apply_range_stragglings()` (in module *tracketch.physics.utils_ions*), 52

`arrival_time_dijkstra()` (in module *tracketch.wavefront*), 55

`arrival_time_dijkstra_fast()` (in module *tracketch.wavefront*), 54

`arrival_time_fmm()` (in module *tracketch.wavefront*), 55

`arrival_time_map` (*tracketch.simulation.simulator.TrackSimulator* attribute), 31

`arrival_time_map` (*tracketch.TrackSimulator* attribute), 18

C

`calculate_RDD_Gy()` (*tracketch.simulation.simulator.TrackSimulator* method), 35

`calculate_RDD_Gy()` (*tracketch.TrackSimulator* method), 20

`convert_map_linspace_to_logspace()` (in module *tracketch.simulation.utils*), 36

`convert_map_logspace_to_linspace()` (in module *tracketch.simulation.utils*), 36

`convert_MeV_to_MeV_u()` (in module *tracketch*), 28

`convert_MeV_to_MeV_u()` (in module *tracketch.physics*), 45

`convert_MeV_to_MeV_u()` (in module *tracketch.physics.utils_ions*), 50

`convert_MeV_u_to_MeV()` (in module *tracketch*), 29

`convert_MeV_u_to_MeV()` (in module *tracketch.physics*), 45

`convert_MeV_u_to_MeV()` (in module *tracketch.physics.utils_ions*), 50

`convert_SRIM_distance_to_um()` (in module *tracketch.physics.SRIM.SRIM_lib*), 57

`convert_SRIM_output_to_df()` (in module *tracketch.physics.SRIM.SRIM_lib*), 57

`create_simulation_grid()` (in module *tracketch.simulation.utils*), 36

`CSDA_range_um` (*tracketch.simulation.simulator.TrackSimulator* attribute), 31

`CSDA_range_um` (*tracketch.TrackSimulator* attribute), 18

D

`debris_alpha` (*tracketch.EtchRateModel* attribute), 27

`debris_beta` (*tracketch.EtchRateModel* attribute), 27

`debris_damping_enabled` (*tracketch.etching.etch_rate_model.EtchRateModel* property), 38

`debris_damping_enabled` (*tracketch.EtchRateModel* property), 23

`default_etch_rate_model()` (in module *tracketch.etching.etch_rate_model_io*), 41

`dose_map` (*tracketch.simulation.simulator.TrackSimulator* attribute), 30

`dose_map` (*tracketch.TrackSimulator* attribute), 18

E

`energy_profile_MeV_u` (*tracketch.simulation.simulator.TrackSimulator* attribute), 31

`energy_profile_MeV_u` (*tracketch.TrackSimulator* attribute), 18

`etch_model` (*tracketch.simulation.simulator.TrackSimulator* attribute), 31

`etch_model` (*tracketch.TrackSimulator* attribute), 19

`etch_rate_map` (*tracketch.simulation.simulator.TrackSimulator* attribute), 31

`etch_rate_map` (*tracketch.TrackSimulator* attribute), 18

`EtchRateModel` (class in *tracketch*), 23

`EtchRateModel` (class in *tracketch.etching.etch_rate_model*), 37

`eval()` (*tracketch.etching.etch_rate_model.EtchRateModel* method), 38

`eval()` (*tracketch.EtchRateModel* method), 23

`eval_uncertainty_band()` (*tracketch.etching.etch_rate_model.EtchRateModel* method), 38

`eval_uncertainty_band()` (*tracketch.EtchRateModel* method), 23

`extrapolation_mode` (*tracketch.EtchRateModel* attribute), 27

G

`generate_SRIM_dfs()` (in module *tracketch.physics.SRIM.SRIM_lib*), 57

`get_arrival_time_map()` (in module *tracketch.wavefront.utils_march*), 56

`get_bragg_correction()` (in module *tracketch.physics.SRIM.SRIM_lib*), 57

`get_CSDA_range_um_libam()` (in module *tracketch.physics.libamtrack*), 48

`get_CSDA_um()` (in module *tracketch*), 28

`get_CSDA_um()` (in module *tracketch.physics*), 43

`get_CSDA_um()` (in module *tracketch.physics.utils_ions*), 50

`get_dose_map_Gy()` (in module *tracketch.physics*), 43

`get_dose_map_Gy()` (in module *tracketch.physics.utils_ions*), 51

`get_energy_after_slab_MeV_u()` (in module *tracketch.physics*), 45

`get_energy_after_slab_MeV_u()` (in module *tracketch.physics.utils_ions*), 50

`get_energy_after_slab_MeV_um_libam()` (in module *tracketch.physics.libamtrack*), 47

`get_f_dose_distribution()` (in module *tracketch.physics.utils_ions*), 53

`get_increments()` (*tracketch.etching.etch_rate_model.EtchRateModel* method), 38

`get_increments()` (*tracketch.EtchRateModel* method), 24

`get_iso_time_contour()` (in module *tracketch.wavefront.utils_march*), 56

`get_iso_time_contour()` (*tracketch.simulation.simulator.TrackSimulator* method), 33

`get_iso_time_contour()` (*tracketch.TrackSimulator* method), 20

`get_iso_time_contour_with_uncertainty()` (*tracketch.simulation.simulator.TrackSimulator* method), 33

`get_iso_time_contour_with_uncertainty()` (*tracketch.TrackSimulator* method), 20

`get_LET_energy_profile()` (in module *tracketch.physics*), 46

`get_LET_energy_profile()` (in module *tracketch.physics.utils_ions*), 52

`get_LET_keV_um()` (in module *tracketch*), 28

`get_LET_keV_um()` (in module *tracketch.physics*), 43

`get_LET_keV_um()` (in module *tracketch.physics.utils_ions*), 48

`get_LET_keV_um_libam()` (in module *tracketch.physics.libamtrack*), 47

`get_log_doses()` (*tracketch.etching.etch_rate_model.EtchRateModel*

method), 39

`get_log_doses()` (*tracketch.EtchRateModel* method), 24

`get_log_velocity_increments()` (*tracketch.etching.etch_rate_model.EtchRateModel* method), 39

`get_log_velocity_increments()` (*tracketch.EtchRateModel* method), 24

`get_material_config()` (in module *tracketch.physics.utils_ions*), 48

`get_material_number_and_density()` (in module *tracketch.physics.libamtrack*), 46

`get_mean_core_value()` (in module *tracketch.physics*), 44

`get_mean_core_value()` (in module *tracketch.physics.utils_ions*), 51

`get_model_path()` (in module *tracketch.etching.etch_rate_model_io*), 41

`get_range_straggling_um()` (in module *tracketch.physics*), 46

`get_range_straggling_um()` (in module *tracketch.physics.utils_ions*), 52

`get_RDD_Gy()` (in module *tracketch*), 27

`get_RDD_Gy()` (in module *tracketch.physics*), 42

`get_RDD_Gy()` (in module *tracketch.physics.utils_ions*), 49

`get_SRIM_df()` (in module *tracketch.physics.SRIM.SRIM_lib*), 57

`get_SRIM_df_folder()` (in module *tracketch.physics.SRIM.SRIM_lib*), 57

`get_SRIM_raw_folder()` (in module *tracketch.physics.SRIM.SRIM_lib*), 57

`get_track_detectability()` (*tracketch.simulation.simulator.TrackSimulator* method), 34

`get_track_detectability()` (*tracketch.TrackSimulator* method), 21

`get_track_length_um()` (*tracketch.simulation.simulator.TrackSimulator* method), 34

`get_track_length_um()` (*tracketch.TrackSimulator* method), 21

`get_track_length_um_with_uncertainty()` (*tracketch.simulation.simulator.TrackSimulator* method), 34

`get_track_length_um_with_uncertainty()` (*tracketch.TrackSimulator* method), 21

`get_track_radius_from_contour()` (in module *tracketch.simulation.utils*), 37

`get_track_radius_um()` (*tracketch.simulation.simulator.TrackSimulator* method), 33

`get_track_radius_um()` (*tracketch.TrackSimulator* method), 21

`get_track_radius_um_with_uncertainty()` (*tracketch.simulation.simulator.TrackSimulator* method), 33

`get_track_radius_um_with_uncertainty()`

(*tracketch.TrackSimulator* method), 22

I

is_monotonic() (*tracketch.etching.etch_rate_model.EtchRateModel* method), 38

is_monotonic() (*tracketch.EtchRateModel* method), 24

is_track_detected() (*tracketch.etch.simulation.simulator.TrackSimulator* method), 35

is_track_detected() (*tracketch.TrackSimulator* method), 22

L

LET_profile_keV_um (*tracketch.etch.simulation.simulator.TrackSimulator* attribute), 31

LET_profile_keV_um (*tracketch.TrackSimulator* attribute), 19

linear_increments_to_log() (*tracketch.etch.etching.etch_rate_model.EtchRateModel* static method), 40

linear_increments_to_log() (*tracketch.EtchRateModel* static method), 24

linear_log_velocity_increments_to_optimization() (*tracketch.etching.etch_rate_model.EtchRateModel* static method), 40

linear_log_velocity_increments_to_optimization() (*tracketch.EtchRateModel* static method), 24

load_etchrate_model() (in module *tracketch*), 27

load_etchrate_model() (in module *tracketch.etching.etch_rate_model_io*), 42

load_from_json() (*tracketch.etching.etch_rate_model.EtchRateModel* class method), 41

load_from_json() (*tracketch.EtchRateModel* class method), 24

load_SRIM_cfg() (in module *tracketch.physics.SRIM.SRIM_lib*), 57

log_increments_to_linear() (*tracketch.etching.etch_rate_model.EtchRateModel* static method), 40

log_increments_to_linear() (*tracketch.EtchRateModel* static method), 25

log_velocity_increments_to_linear() (*tracketch.etching.etch_rate_model.EtchRateModel* static method), 40

log_velocity_increments_to_linear() (*tracketch.EtchRateModel* static method), 25

LOW_DOSE_ANCHOR_GY (*tracketch.etching.etch_rate_model.EtchRateModel* attribute), 37

LOW_DOSE_ANCHOR_GY (*tracketch.EtchRateModel* attribute), 23

M

module

tracketch, 17

tracketch.etching.etch_rate_model, 37

tracketch.etching.etch_rate_model_io, 41

tracketch.physics, 42

tracketch.physics.libamtrack, 46

tracketch.physics.SRIM.SRIM_lib, 57

tracketch.physics.utils_ions, 48

tracketch.simulation.plots, 35

tracketch.simulation.simulator, 29

tracketch.simulation.utils, 36

tracketch.wavefront, 54

tracketch.wavefront.utils_march, 56

N

name (*tracketch.EtchRateModel* attribute), 27

P

plot() (*tracketch.etching.etch_rate_model.EtchRateModel* method), 40

plot() (*tracketch.EtchRateModel* method), 25

plot_iso_time_contour() (*tracketch.simulation.simulator.TrackSimulator* method), 35

plot_iso_time_contour() (*tracketch.TrackSimulator* method), 22

plot LET energy profiles() (in module *tracketch.simulation.plots*), 36

plot LET energy profiles() (*tracketch.simulation.simulator.TrackSimulator* method), 35

plot LET energy profiles() (*tracketch.TrackSimulator* method), 22

plot_map() (in module *tracketch.simulation.plots*), 35

plot_map() (*tracketch.simulation.simulator.TrackSimulator* method), 35

plot_map() (*tracketch.TrackSimulator* method), 22

plot_track_radius() (*tracketch.simulation.simulator.TrackSimulator* method), 35

plot_track_radius() (*tracketch.TrackSimulator* method), 22

R

RDD_integral() (in module *tracketch.physics*), 44

RDD_integral() (in module *tracketch.physics.utils_ions*), 49

RDD_libamtrack_Gy() (in module *tracketch.physics.libamtrack*), 47

recalculate_from_current_etch_model() (*tracketch.simulation.simulator.TrackSimulator* method), 32

recalculate_from_current_etch_model() (*tracketch.TrackSimulator* method), 23

S

save_etchrate_model() (in module *tracketch*), 27

`save_etchrate_model()` (in module `track-etch.etching.etch_rate_model_io`), 42

`save_to_json()` (`track-etch.etching.etch_rate_model.EtchRateModel` method), 41

`save_to_json()` (`tracketch.EtchRateModel` method), 25

`set_bragg_correction()` (in module `track-etch.physics.SRIM.SRIM_lib`), 57

`SRIM_CSDA_um_to_MeV_u()` (in module `track-etch.physics.SRIM.SRIM_lib`), 57

`SRIM_LET_to_MeV_u()` (in module `track-etch.physics.SRIM.SRIM_lib`), 57

`SRIM_MeV_u_to_CSDA_um()` (in module `track-etch.physics.SRIM.SRIM_lib`), 57

`SRIM_MeV_u_to_LET()` (in module `track-etch.physics.SRIM.SRIM_lib`), 57

`summary()` (`tracketch.etching.etch_rate_model.EtchRateModel` method), 37

`summary()` (`tracketch.EtchRateModel` method), 26

T

`tracketch`
module, 17

`tracketch.etching.etch_rate_model`
module, 37

`tracketch.etching.etch_rate_model_io`
module, 41

`tracketch.physics`
module, 42

`tracketch.physics.libamtrack`
module, 46

`tracketch.physics.SRIM.SRIM_lib`
module, 57

`tracketch.physics.utils_ions`
module, 48

`tracketch.simulation.plots`
module, 35

`tracketch.simulation.simulator`
module, 29

`tracketch.simulation.utils`
module, 36

`tracketch.wavefront`
module, 54

`tracketch.wavefront.utils_march`
module, 56

`TrackSimulator` (class in `tracketch`), 17

`TrackSimulator` (class in `track-etch.simulation.simulator`), 29

U

`update_debris_params()` (`track-etch.etching.etch_rate_model.EtchRateModel` method), 37

`update_debris_params()` (`tracketch.EtchRateModel` method), 26

`update_dose_anchors()` (`track-etch.etching.etch_rate_model.EtchRateModel` method), 39

`update_dose_anchors()` (`tracketch.EtchRateModel` method), 26

`update_etch_model_and_recalculate()` (`track-etch.simulation.simulator.TrackSimulator` method), 32

`update_etch_model_and_recalculate()` (`track-etch.TrackSimulator` method), 23

`update_from_increments()` (`track-etch.etching.etch_rate_model.EtchRateModel` method), 39

`update_from_increments()` (`track-etch.EtchRateModel` method), 26

`update_from_log_velocity_increments()` (`tracketch.etching.etch_rate_model.EtchRateModel` method), 39

`update_from_log_velocity_increments()` (`tracketch.EtchRateModel` method), 26

`update_velocities()` (`track-etch.etching.etch_rate_model.EtchRateModel` method), 38

`update_velocities()` (`tracketch.EtchRateModel` method), 26

V

`V_bulk_um_h` (`tracketch.EtchRateModel` attribute), 26

`V_max_um_h` (`tracketch.EtchRateModel` attribute), 27

`V_max_uncertainty_um_h` (`tracketch.EtchRateModel` attribute), 27